

Christof Teuscher

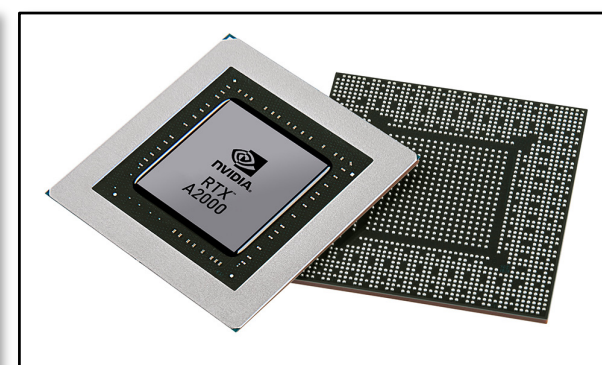
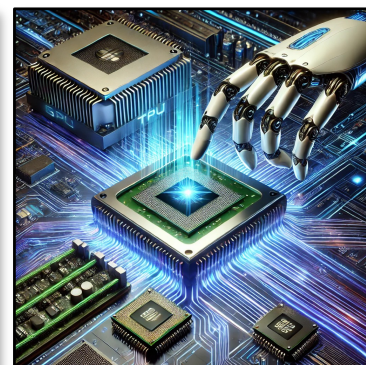
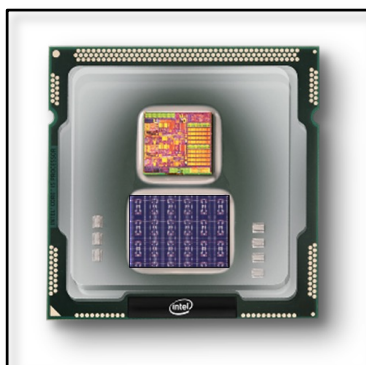
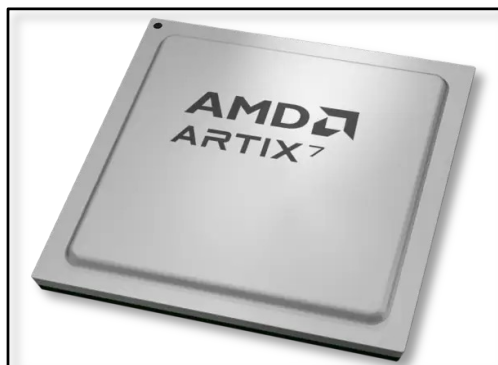
ECE 410/510: Hardware for AI and ML, Spring 2026

GPU Architecture, CUDA, and the Memory Wall

Portland State University
Department of Electrical and Computer Engineering (ECE)

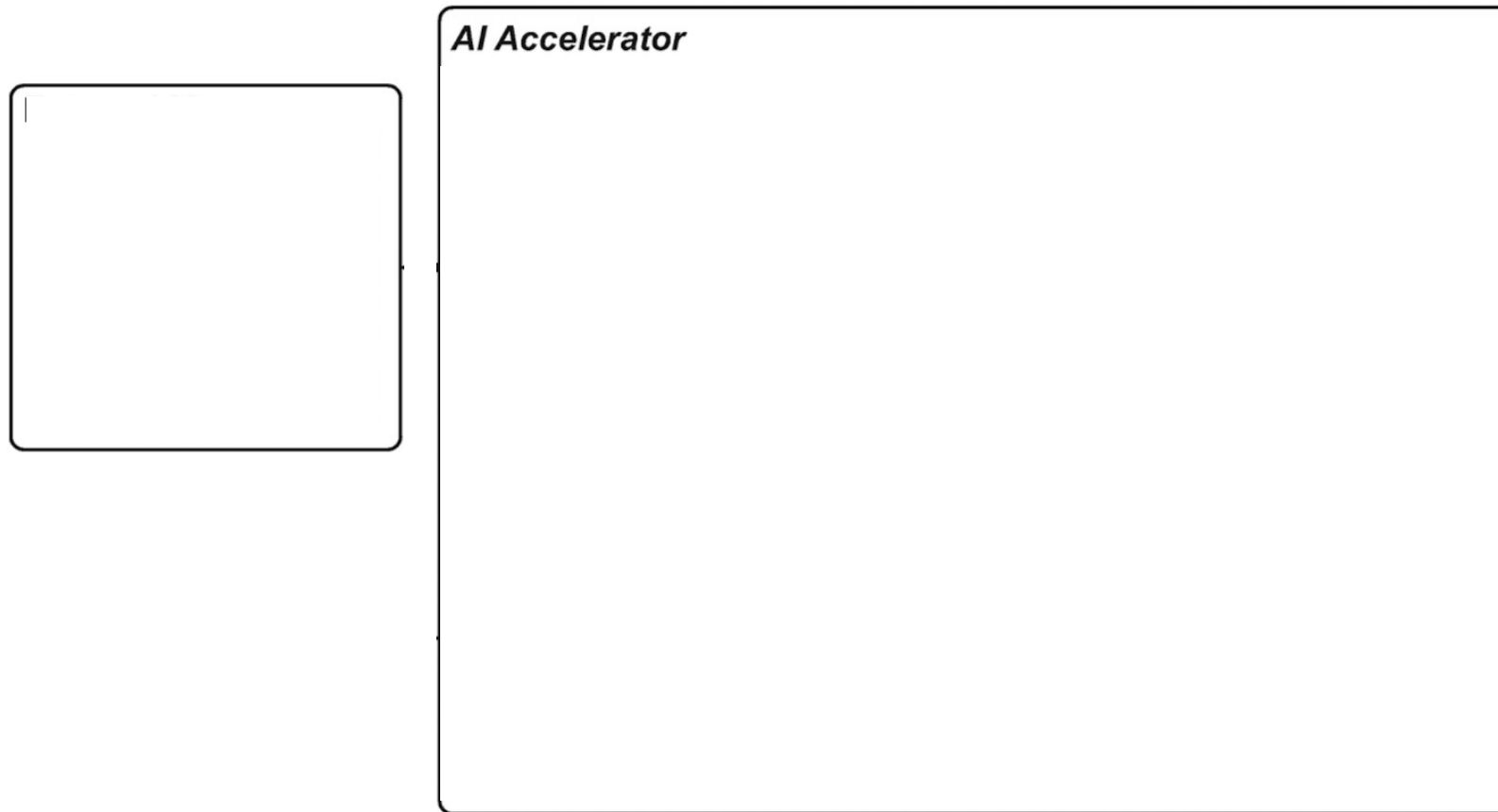
www.teuscher-lab.com

teuscher@pdx.edu



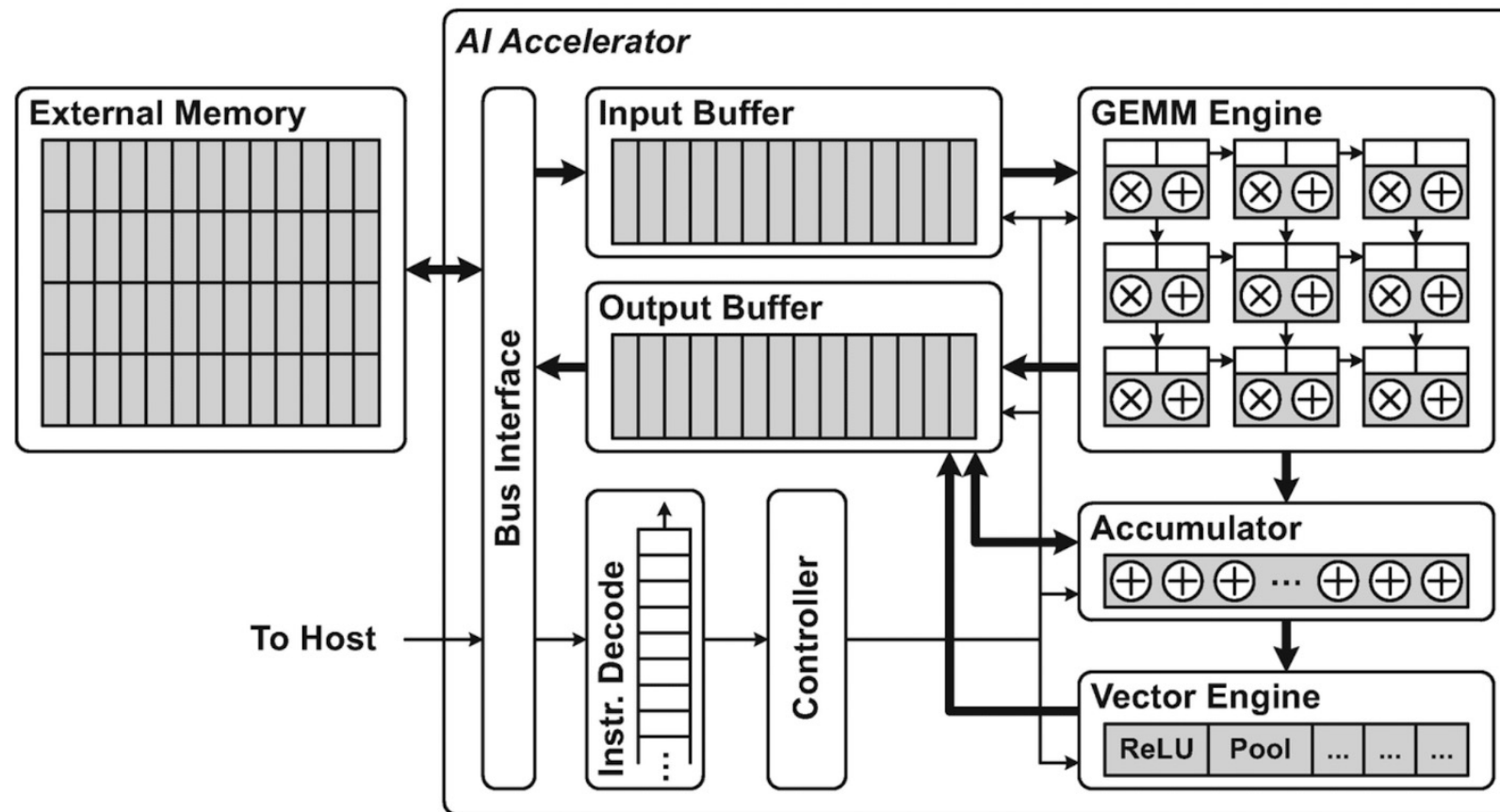


What are the components of a generic AI accelerator?



Where are potential bottlenecks?

General AI accelerator architecture



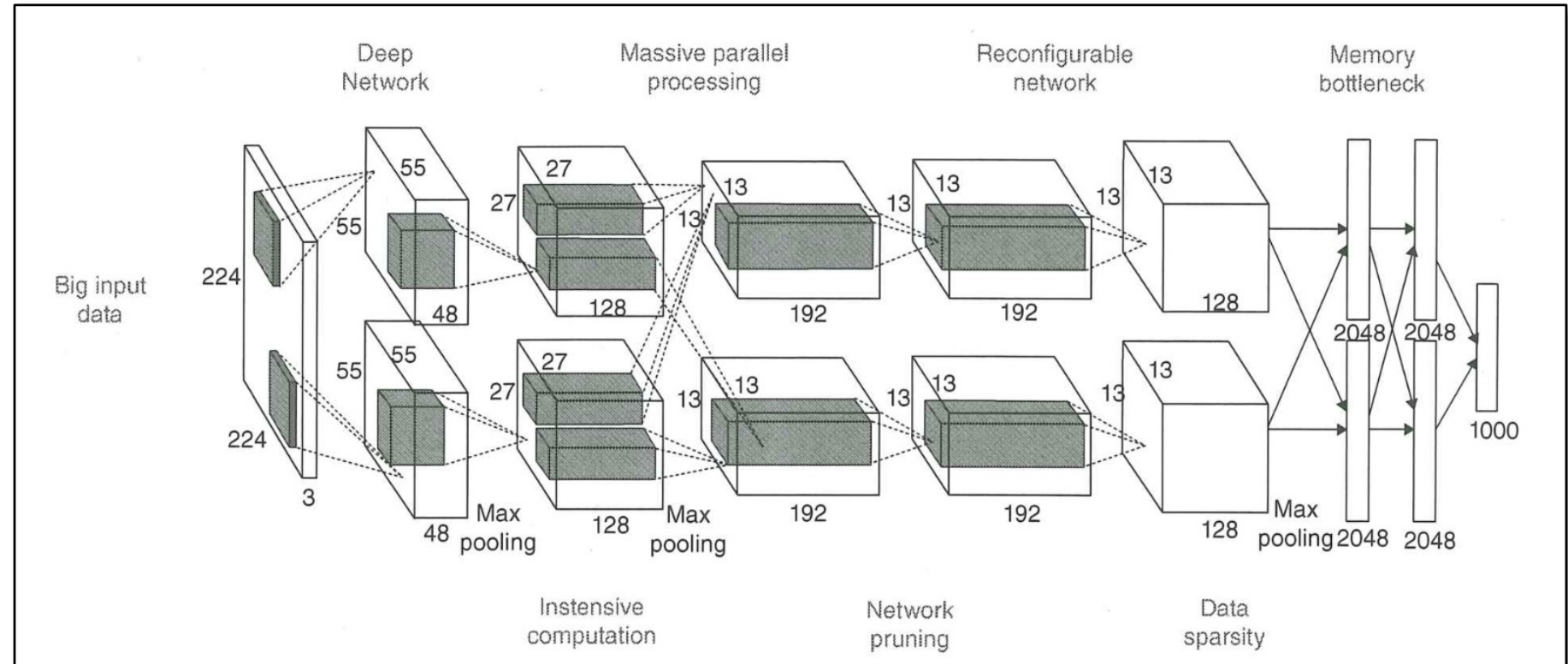
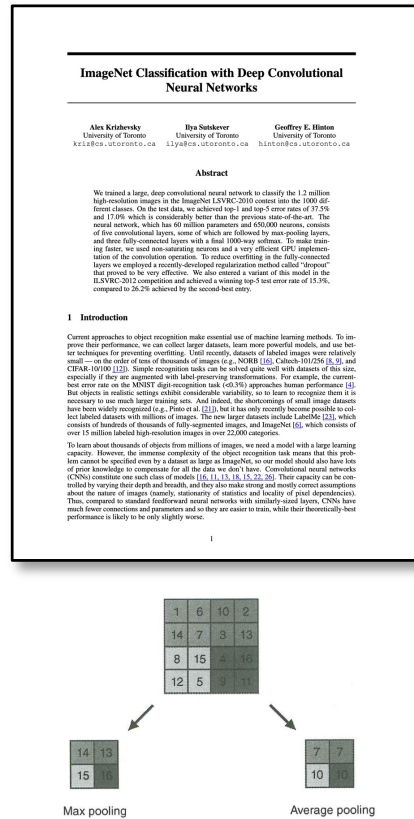
GEMM = General Matrix Multiply

Tensor cores!

A vector is a 1st-order tensor
A matrix is a 2nd-order tensor.

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

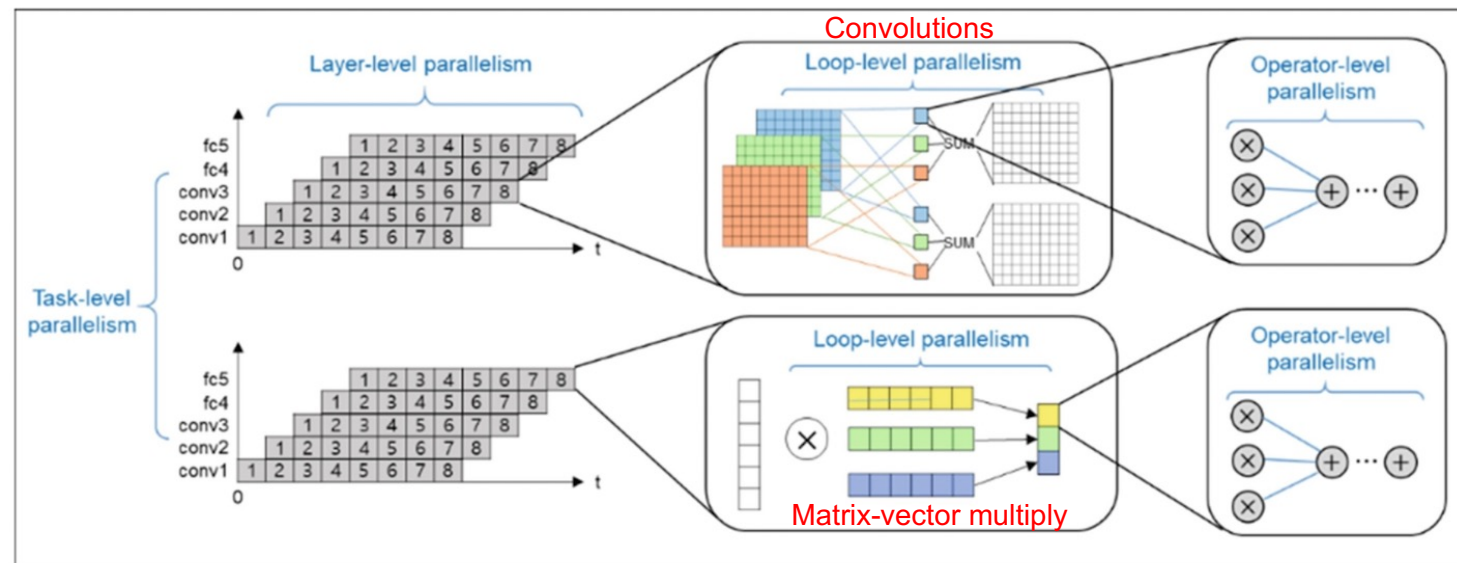
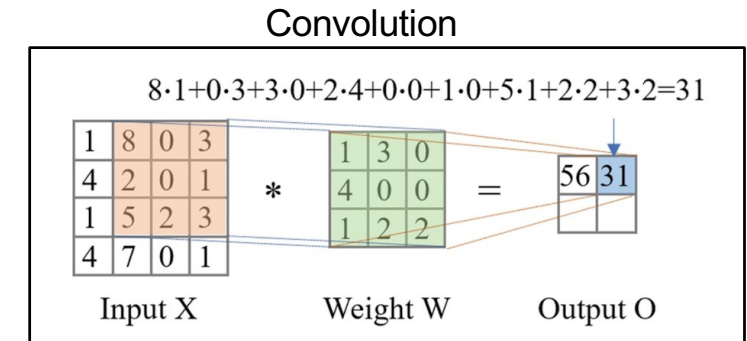
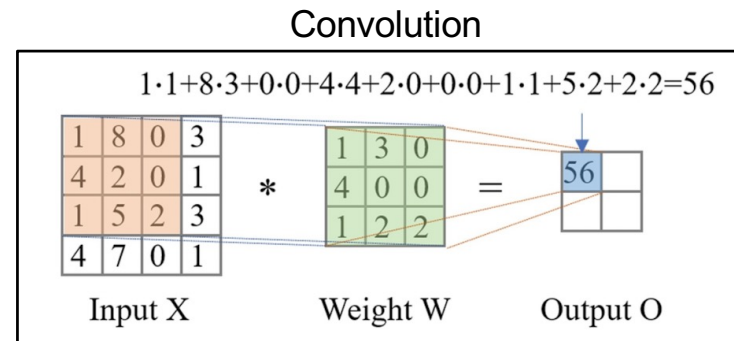
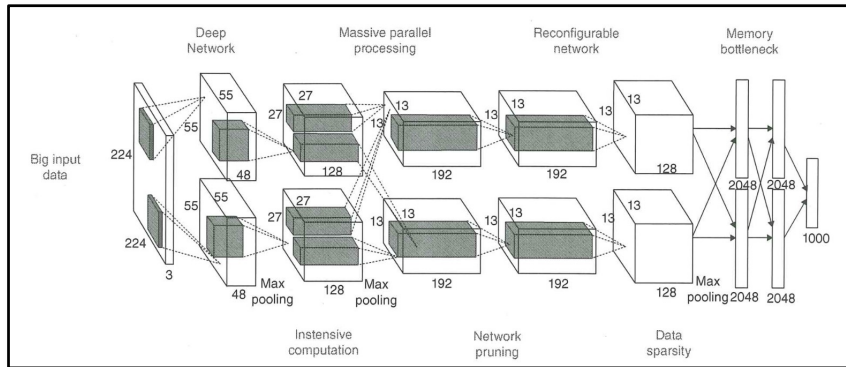
AlexNet: A classical deep convolutional neural network



AlexNet paper: https://papers.nips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

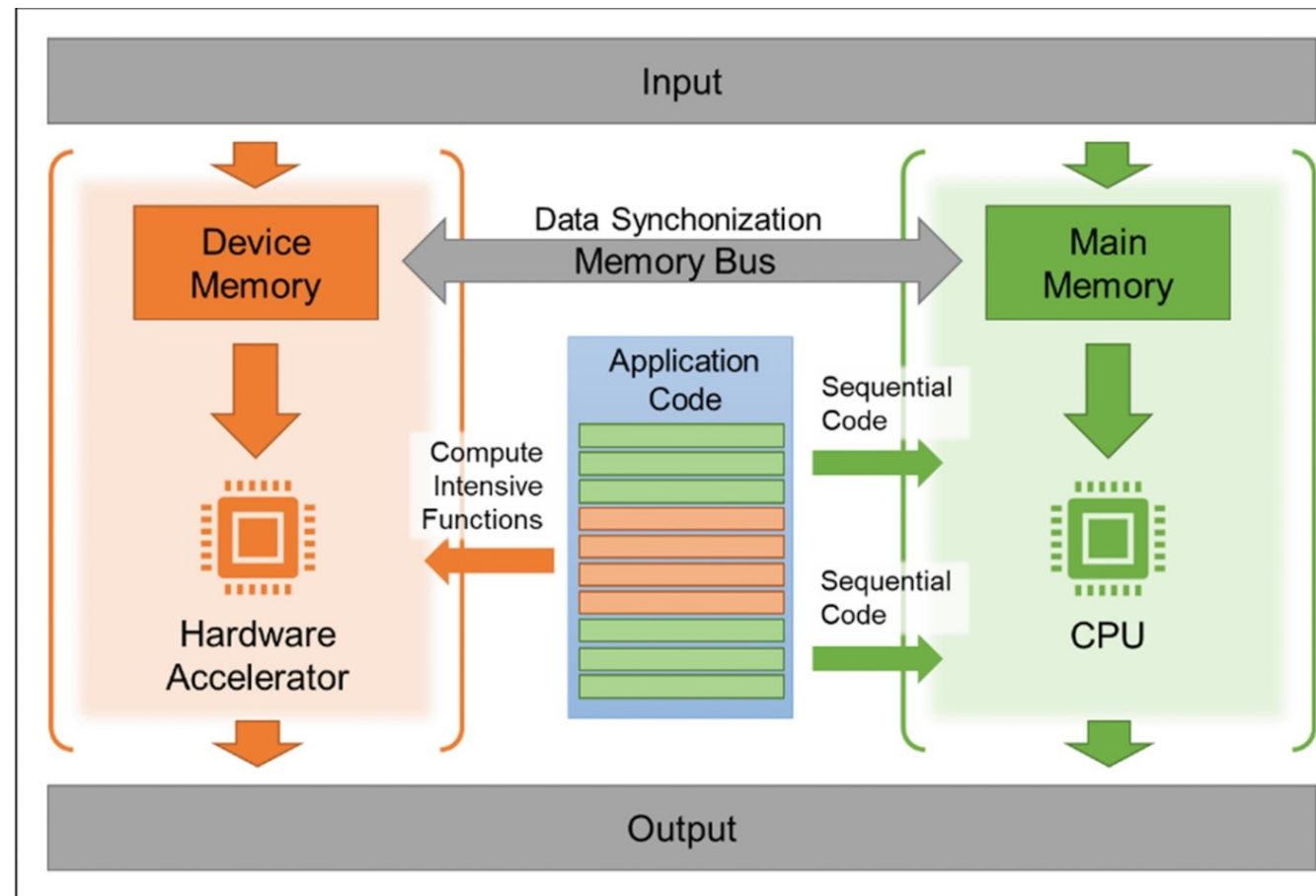
Liu & Law, Artificial Intelligence Hardware Design, 2021

What parallelism can be exploited?



- Max and avg. pooling are not convolutions.
- Similar operation, but convolutions have learnable parameters (weights).

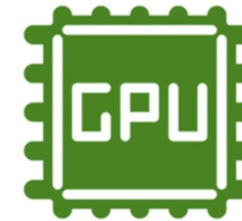
General co-processing mechanism



Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

Hardware for AI/ML

Processor	Power Consumption	Strengths	Limitations
CPU	High	→ Flexible → General-purpose processing → Complex instructions and tasks → System management	→ Possible memory access bottlenecks → Few cores (4–16)
GPU	High	→ Parallel cores (~1000 s of cores) → High-performance AI processing	→ Power consumption → Large footprint
FPGA	Medium	→ Configurable logic gates → Flexible → In-field re-programmability	→ Programming complexity
ASIC	Low	→ Custom logic designed with libraries → Faster processing → Small footprint	→ Fixed function → Expensive custom design
Vision Processing Unit (VPU)	Ultra-low	→ Dedicated image and vision co-processor → Small footprint	→ Limited dataset and batch size → Limited network support
Tensor Processing Unit (TPU)	Low to medium	→ Specialized tool support → Optimized for Tensor-Flow	→ Proprietary design → Limited framework support



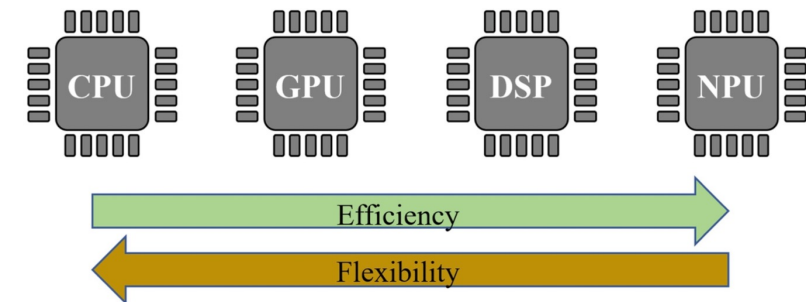
- Domain programmable
- Partially general-purpose
- Parallel workloads
- Power-hungry



- Hardware programmable
- Partially general-purpose
- Any workloads
- High performance
- Low power



- Hard wired
- Special-purpose
- Dedicated workloads
- High performance
- Low power





Christof Teuscher



teuscher@pdx.edu

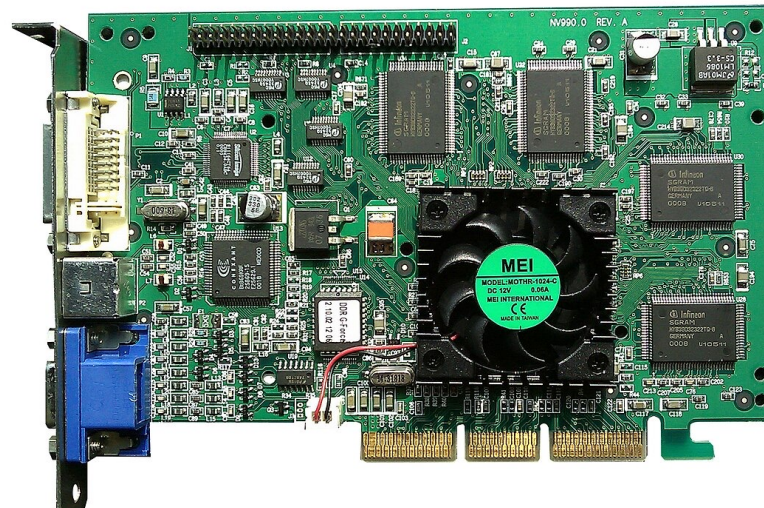
www.teuscher-lab.com/teaching



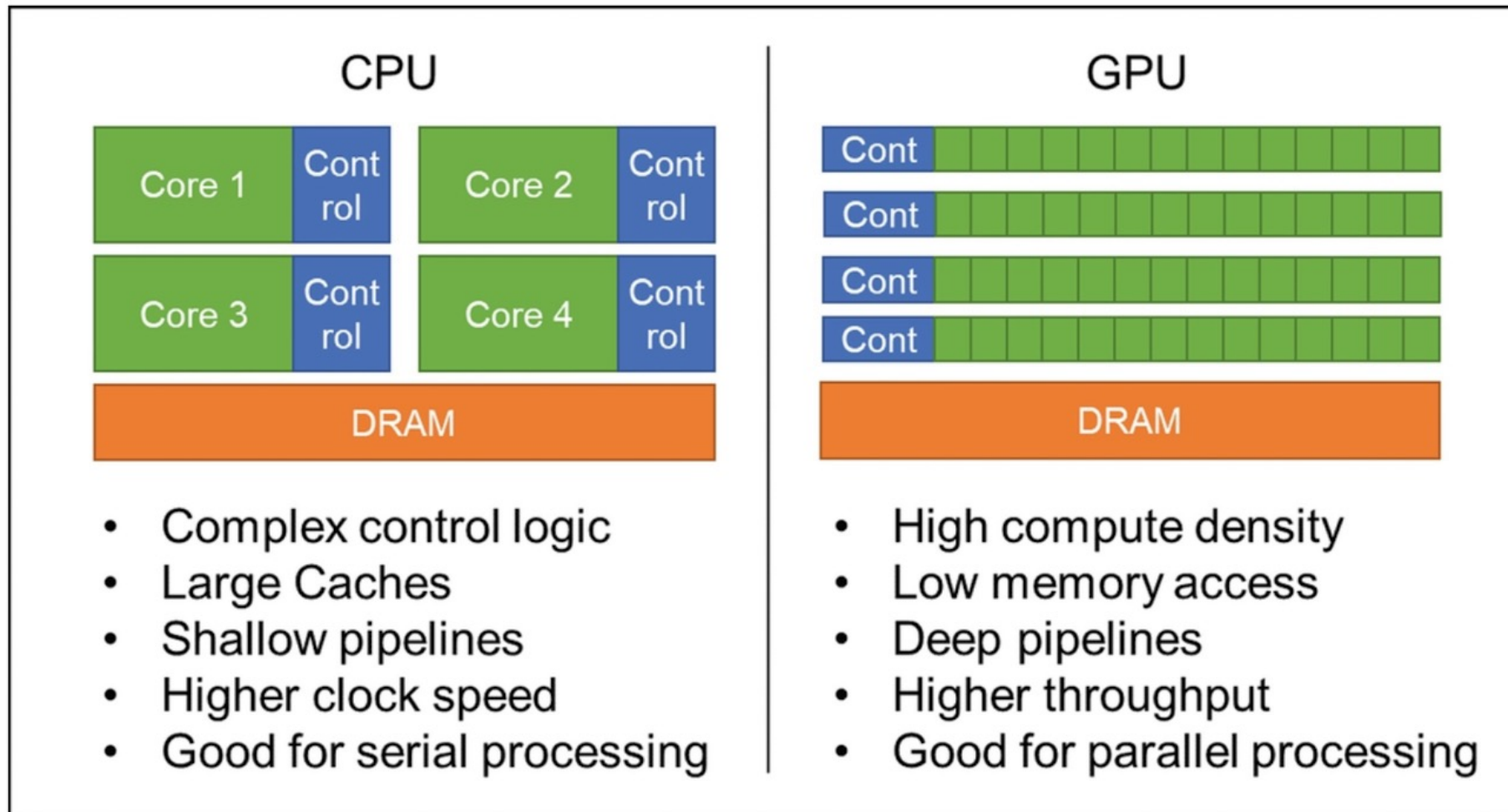
Graphics Processing Units (GPUs)

A bit of history

- Over several decades, GPUs evolved from a single core, fixed function hardware used for graphics solely, to a set of programmable parallel cores.
- The history of modern GPUs starts in 1995 with the introduction of the first 3D add-in cards.
- 1996: **3DFx's Voodoo** graphics card took over about 85% of the market. Cards that could only render 2D became obsolete very fast.
- 1999: **NVIDIA's GeForce 256**, the “world's first GPU.”



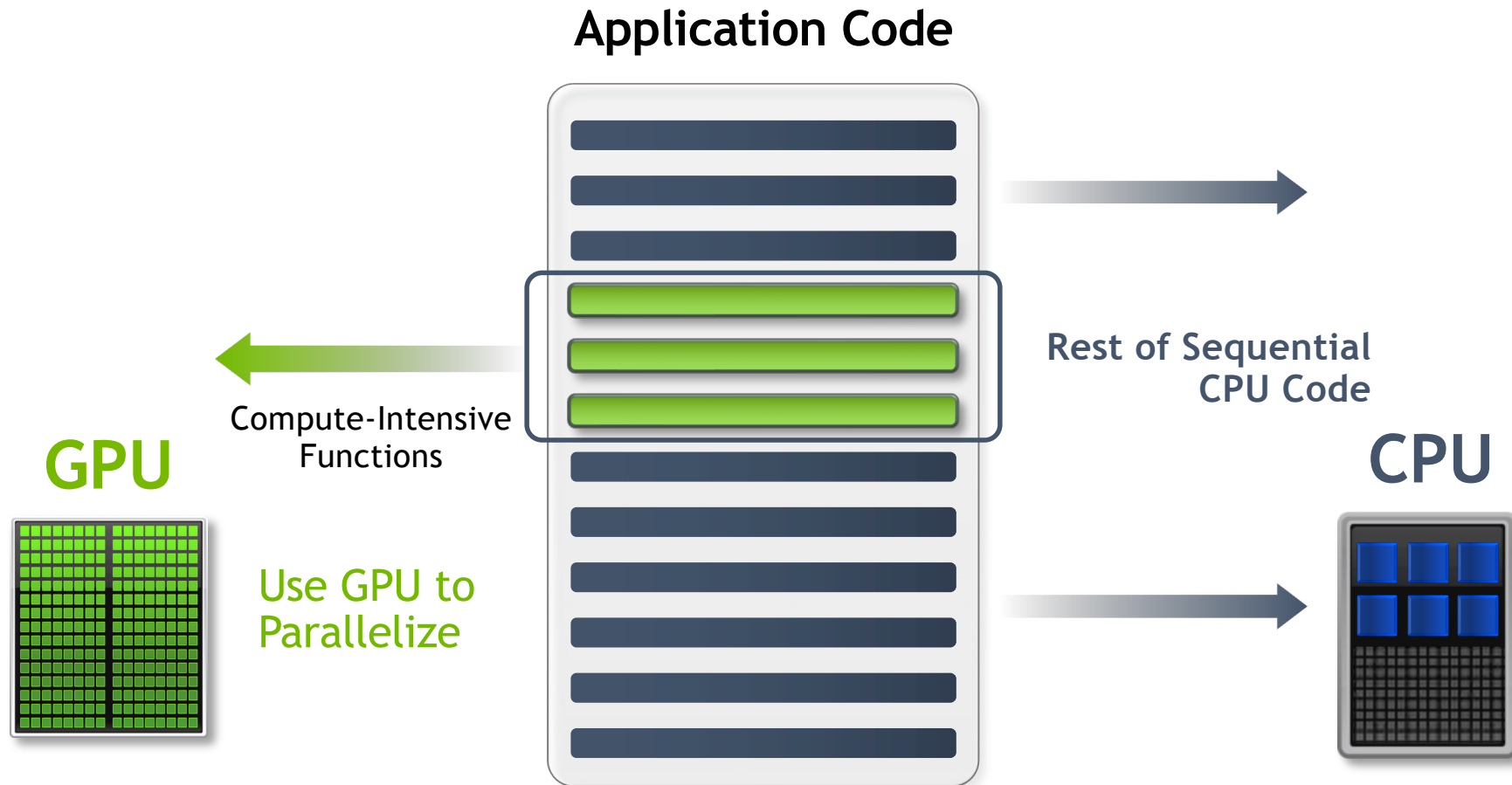
Multi-core vs GPU



SIMT (Single Instruction, Multiple Threads) is a parallel processing model used in GPUs where multiple threads execute the same instruction simultaneously, but on different data.

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

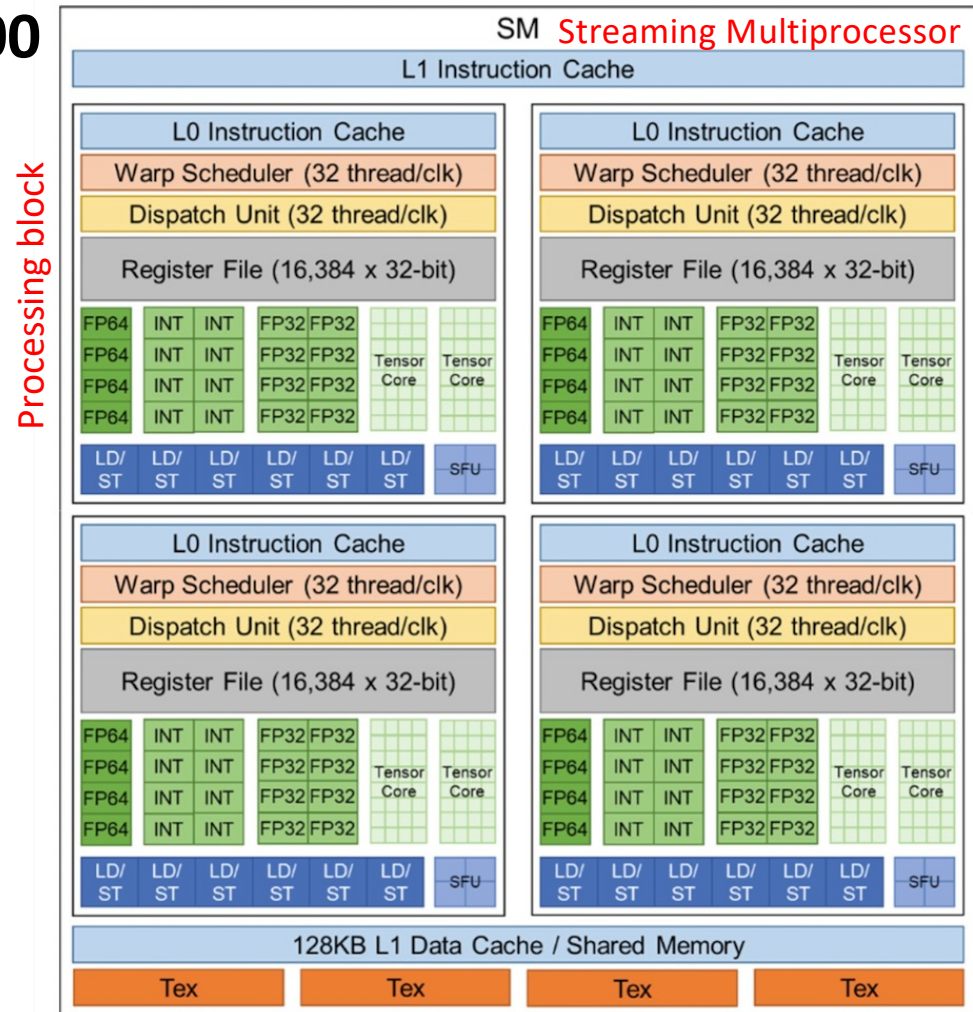
Small Changes, Big Speed-up



NVIDIA

NVIDIA Volta Microarchitecture (2013-2017)

V100



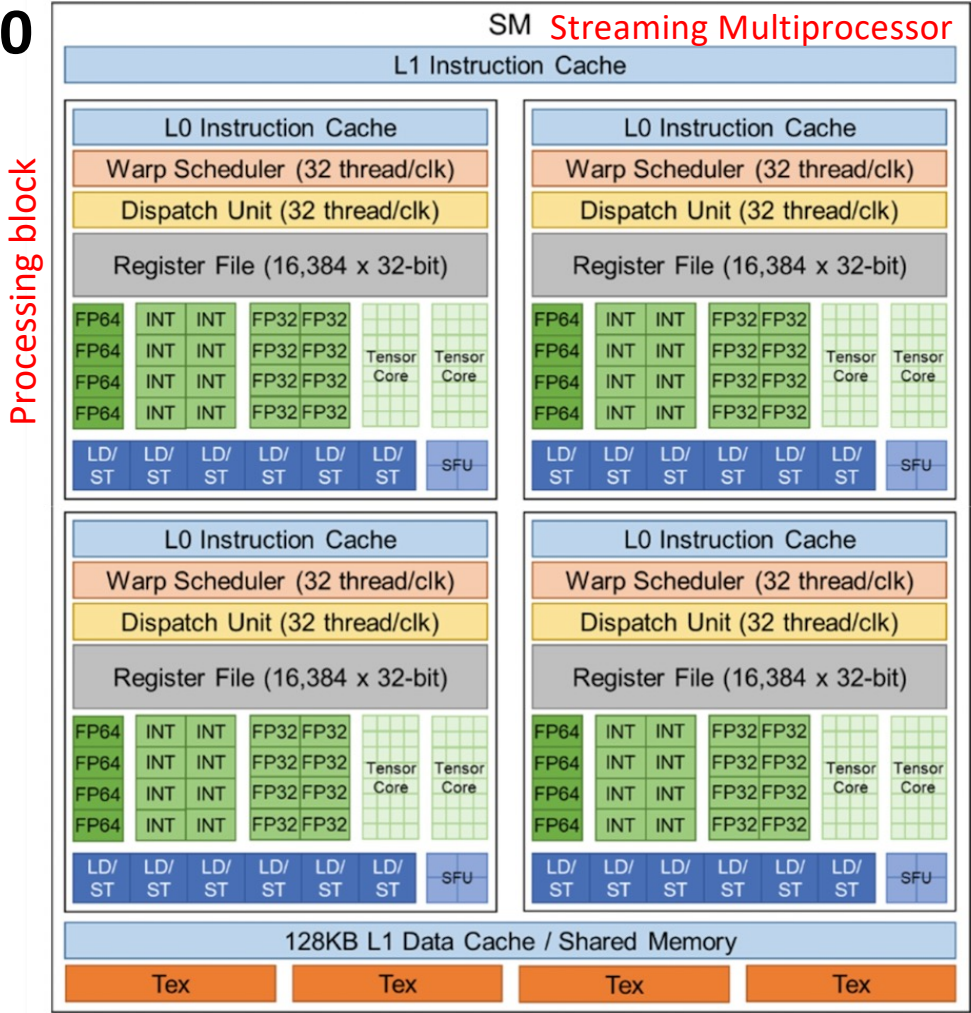
Tex = Texture units
SFU = Special Function Unit

Spec	P100	V100	A100	H100	H200	B100	B200	B300
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell	Blackwell	Blackwell Ultra
Release year	2016	2017	2020	2022	2024	2025	2025	2025
Process node	16nm	12nm	7nm	4nm	4nm	4NP	4NP	4NP
Die config	Single	Single	Single	Single	Single	Dual	Dual	Dual
FP32 TFLOPS	9.3	14	19.5	67	67	45	75	~90
FP16 TFLOPS	18.7	28	77.6	989	1,979	1,750	2,250	~3,500
BF16 TFLOPS	—	—	77.6	989	1,979	1,750	2,250	~3,500
FP8 TFLOPS	—	—	—	1,979	3,958	3,500	4,500	~7,000
FP4 TOPS	—	—	—	—	—	—	9,000	15,000
INT8 TOPS	—	—	624	1,979	3,958	7,000	9,000	—
Tensor cores	—	1st gen	3rd gen	4th gen	4th gen	5th gen	5th gen	5th gen
HBM gen	HBM2	HBM2	HBM2e	HBM3	HBM3e	HBM3e	HBM3e	HBM3e
HBM capacity	16 GB	16/32 GB	40/80 GB	80 GB	141 GB	192 GB	192 GB	288 GB
Memory bandwidth	732 GB/s	900 GB/s	2 TB/s	3.35 TB/s	4.8 TB/s	8 TB/s	8 TB/s	8 TB/s
NVLink bandwidth	160 GB/s	300 GB/s	600 GB/s	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s	1.8 TB/s
TDP	300 W	300 W	400 W	700 W	700 W	700 W	1,000 W	1,400 W

Why not 4x
more
throughput?

NVIDIA Volta Microarchitecture (2013-2017)

V100



Spec	P100	V100	A100	H100	H200	B100	B200	B300
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell	Blackwell	Blackwell Ultra
Release year	2016	2017	2020	2022	2024	2025	2025	2025
Process node	16nm	12nm	7nm	4nm	4nm	4NP	4NP	4NP
Die config	Single	Single	Single	Single	Single	Dual	Dual	Dual
FP32 TFLOPS	9.3	14	19.5	67	67	45	75	~90
FP16 TFLOPS	18.7	28	77.6	989	1,979	1,750	2,250	~3,500
BF16 TFLOPS	—	—	77.6	989	1,979	1,750	2,250	~3,500
FP8 TFLOPS	—	—	—	1,979	3,958	3,500	4,500	~7,000
FP4 TOPS	—	—	—	—	—	—	9,000	15,000
INT8 TOPS	—	—	624	1,979	3,958	7,000	9,000	—
Tensor cores	—	1st gen	3rd gen	4th gen	4th gen	5th gen	5th gen	5th gen
HBM gen	HBM2	HBM2	HBM2e	HBM3	HBM3e	HBM3e	HBM3e	HBM3e
HBM capacity	16 GB	16/32 GB	40/80 GB	80 GB	141 GB	192 GB	192 GB	288 GB
Memory bandwidth	732 GB/s	900 GB/s	2 TB/s	3.35 TB/s	4.8 TB/s	8 TB/s	8 TB/s	8 TB/s
NVLink bandwidth	160 GB/s	300 GB/s	600 GB/s	900 GB/s	900 GB/s	1.8 TB/s	1.8 TB/s	1.8 TB/s
TDP	300 W	300 W	400 W	700 W	700 W	700 W	1,000 W	1,400 W

CUDA core TP
Different execution units.
See diagram!
Tensor core TP

H100 SM architecture (Hopper, 2022)

□ Key H100 SXM5 specifications

- 132 SMs; 16,896 CUDA cores; 528 4th-gen Tensor Cores
- 80 GB HBM3; 3.35 TB/s memory bandwidth
- 228 KB shared memory per SM (L1/shared configurable)
- Peak FP32: 67 TFLOP/s; Peak FP8 (Tensor): 3,958 TFLOP/s

□ SM internal hierarchy

- Each SM has 4 processing blocks; each block has 1 warp scheduler + 32 FP32 cores
- Tensor Cores sit alongside the FP32 units and execute 4x4 MMA in one clock
- Register file: 65,536 x 32-bit registers per SM; shared across all active warps

□ What's the ridge point?

H100 SM architecture (Hopper, 2022)

□ Key H100 SXM5 specifications

- 132 SMs; 16,896 CUDA cores; 528 4th-gen Tensor Cores
- 80 GB HBM3; **3.35 TB/s** memory bandwidth
- 228 KB shared memory per SM (L1/shared configurable)
- Peak FP32: **67 TFLOP/s**; Peak FP8 (Tensor): 3,958 TFLOP/s

□ SM internal hierarchy

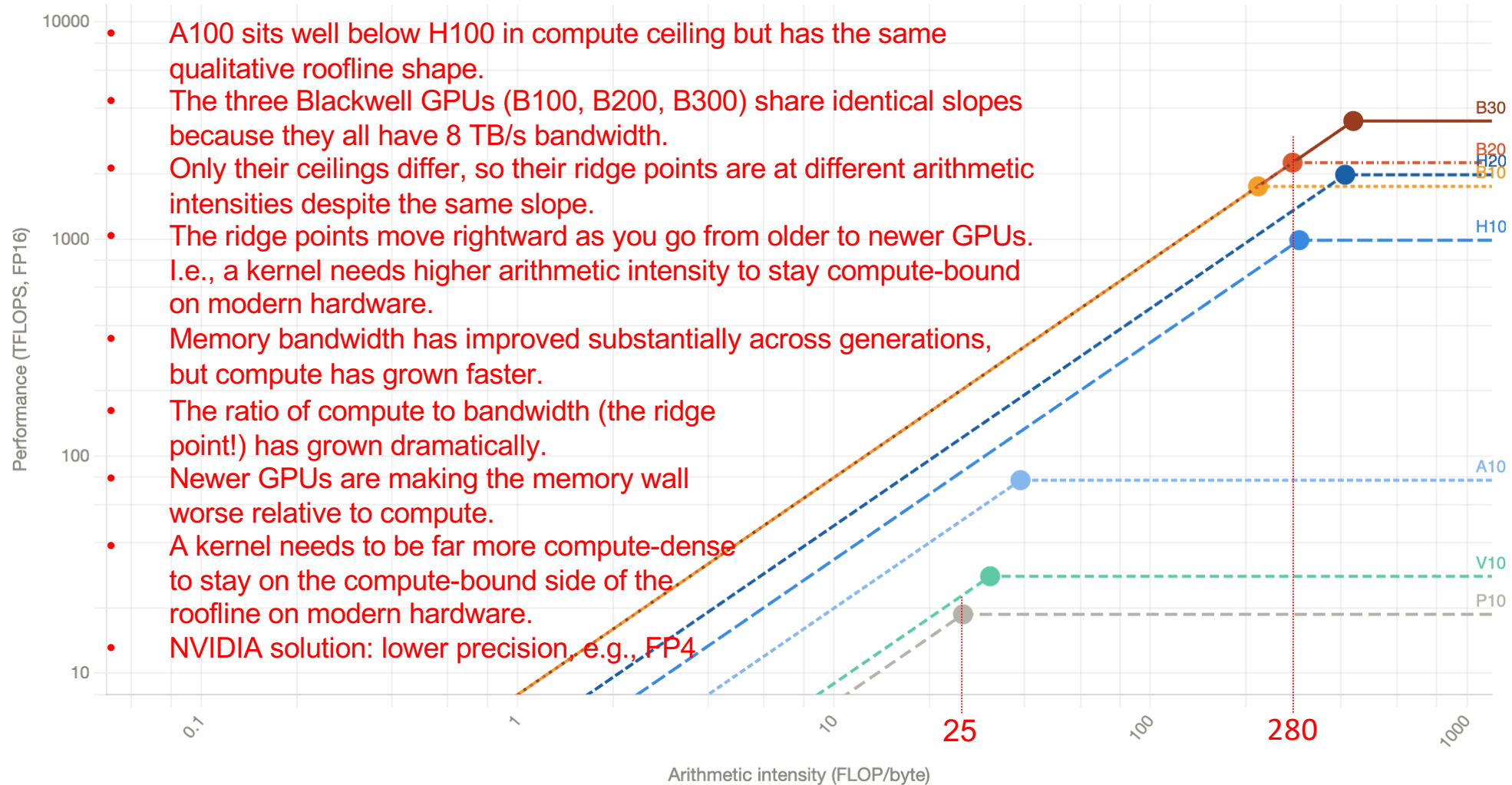
- Each SM has 4 processing blocks; each block has 1 warp scheduler + 32 FP32 cores
- Tensor Cores sit alongside the FP32 units and execute 4x4 MMA in one clock.
- Register file: 65,536 x 32-bit registers per SM; shared across all active warps

□ **Ridge point = Peak FLOP/s / Peak bandwidth = 67 / 3.35 $\approx$ 20 FLOP/byte**

Rooflines (FP16)

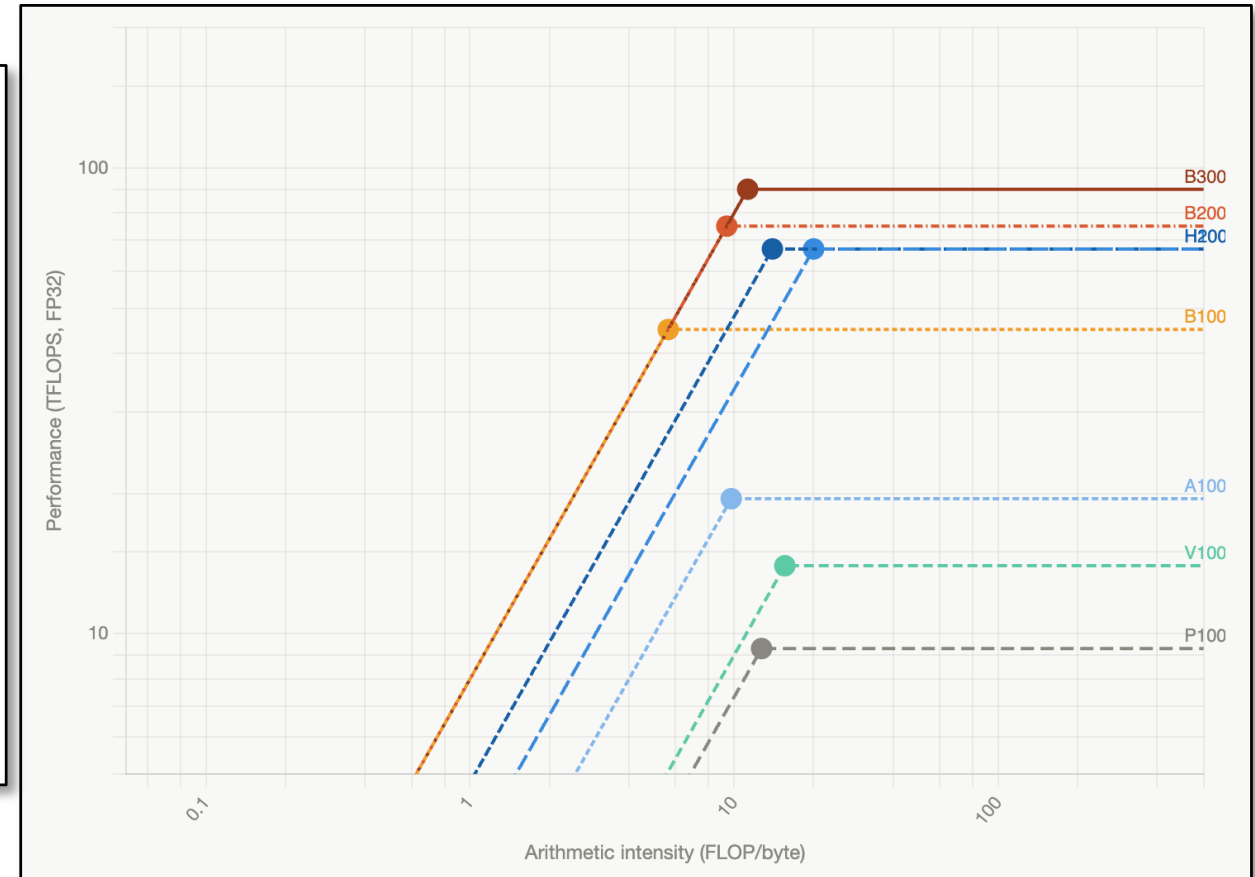
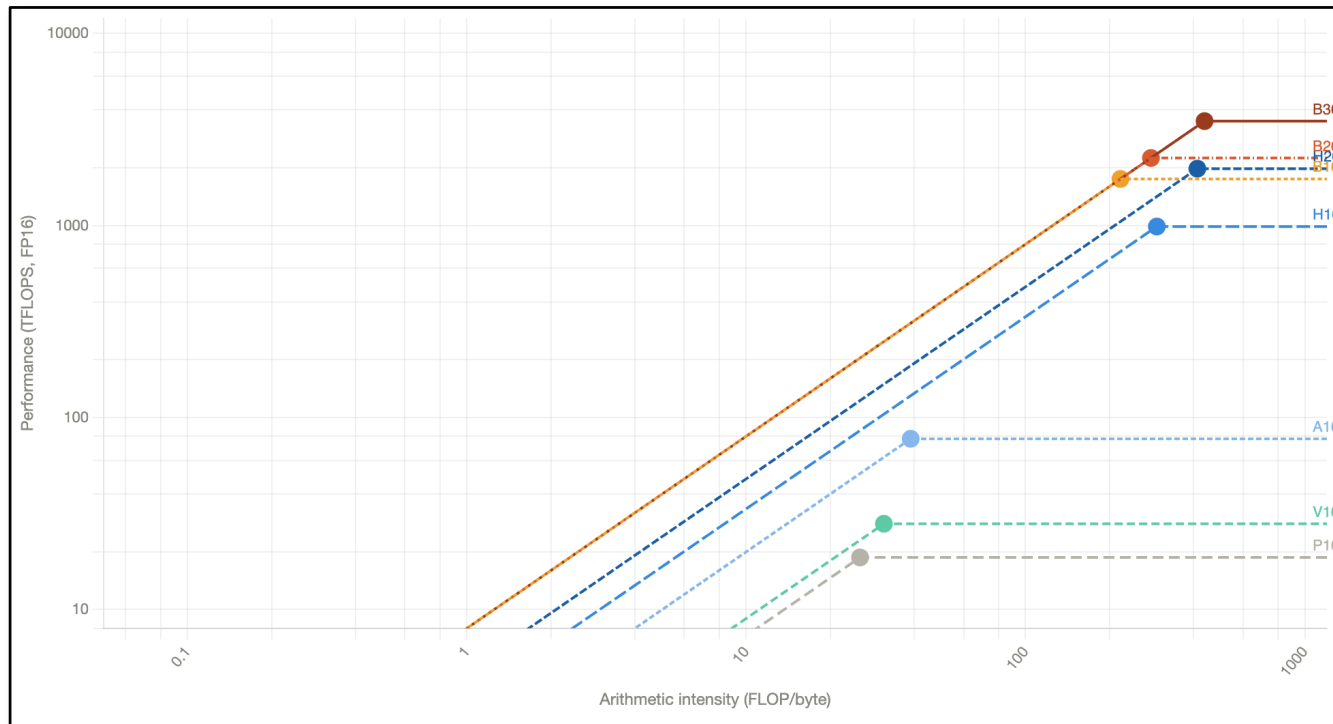


Rooflines (FP16)



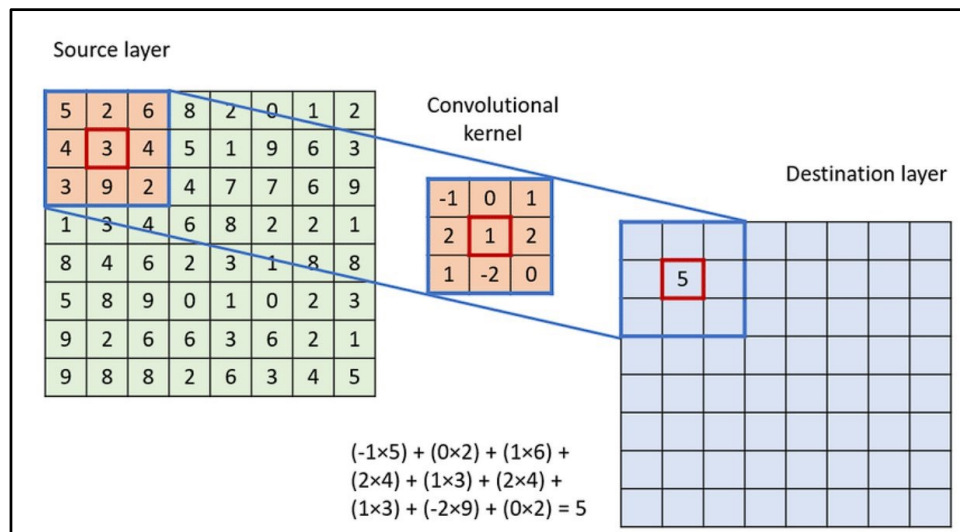
Rooflines (FP32)

What changed compared to FP16? Why?



Tensor cores (1)

- Each tensor core executes MMA (matrix multiplication and accumulation, $D = A \times B + C$) on a 4×4 floating-point matrix.
- They are built to enhance deep learning by boosting matrix arithmetic.
- Low-precision operations often supported.



Convolution

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

FP16 or FP32 FP16 FP16 FP16 or FP32

MMA

Tensor cores (2)

Tensor Cores operate on FP16 input data with FP32 accumulation. The FP16 multiply results in a full precision product that is then accumulated using FP32 addition with the other intermediate products for a 4x4x4 matrix multiply (see Figure 9). In practice, Tensor Cores are used to perform much larger 2D or higher dimensional matrix operations, built up from these smaller elements.

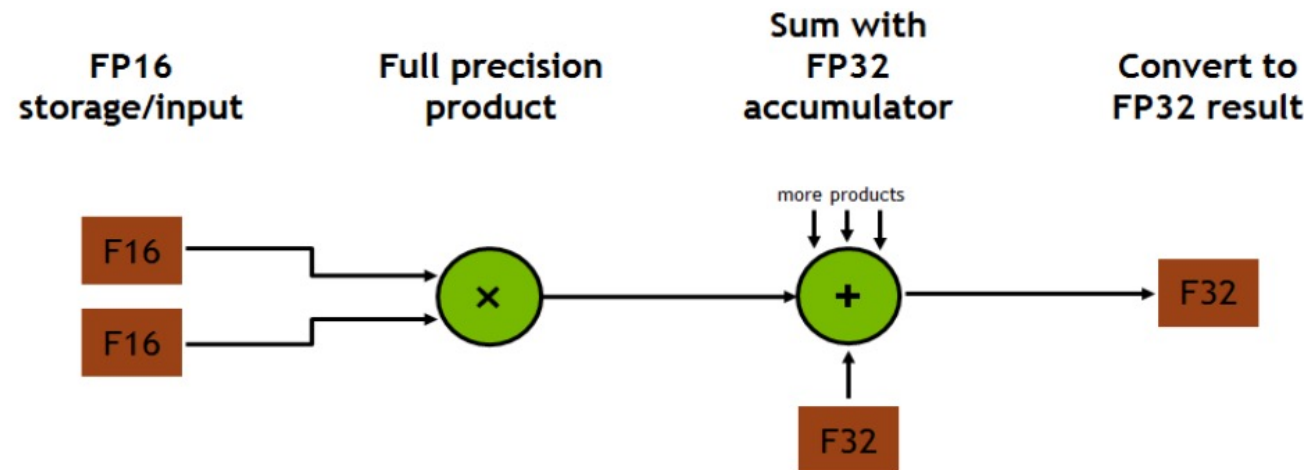
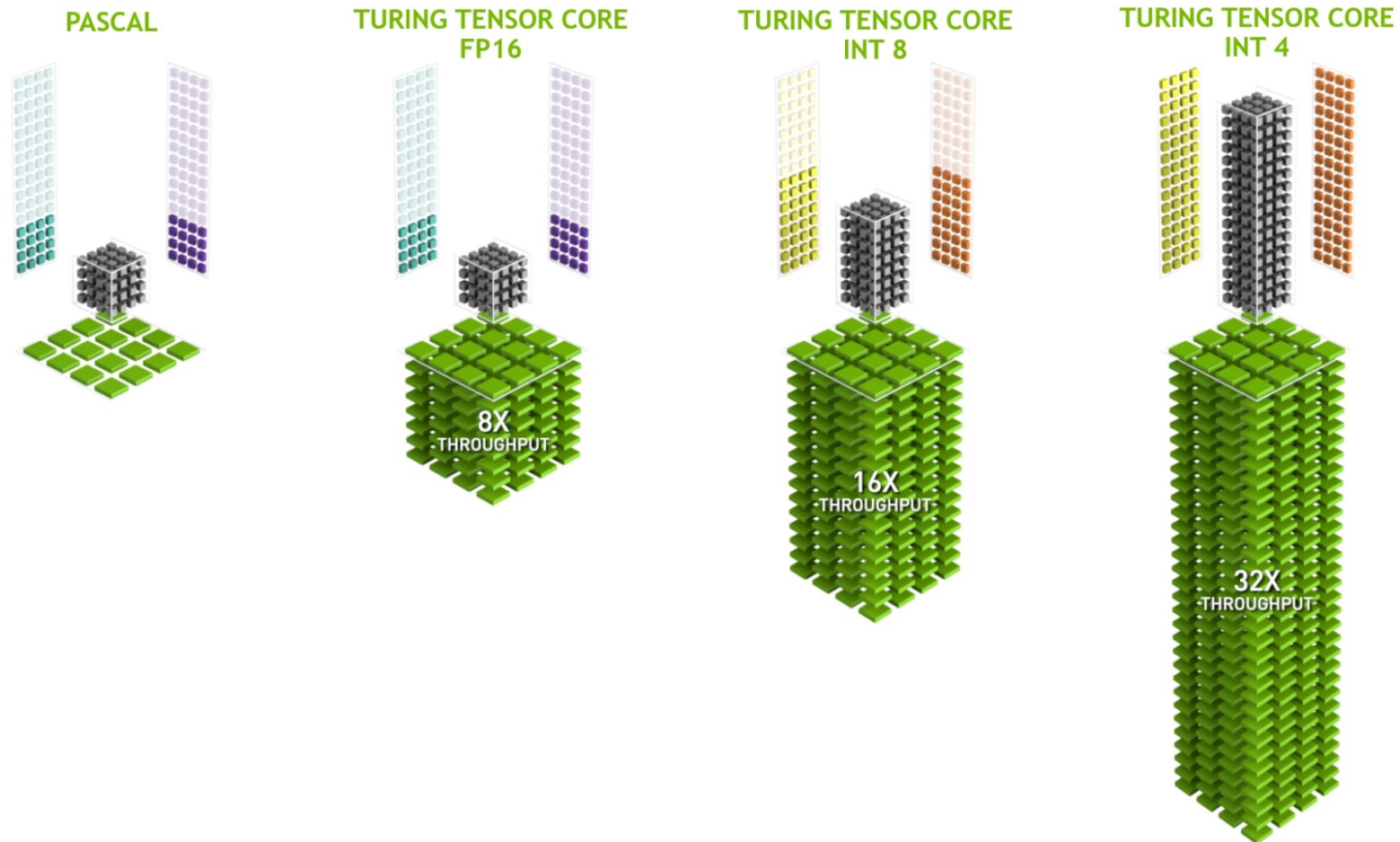


Figure 9. Mixed Precision Multiply and Accumulate in Tensor Core

Tensor cores (3)



New Turing Tensor Cores Provide Multi-Precision for AI Inference.

Streaming vs tensor cores

SM Cores (Streaming Multiprocessors)

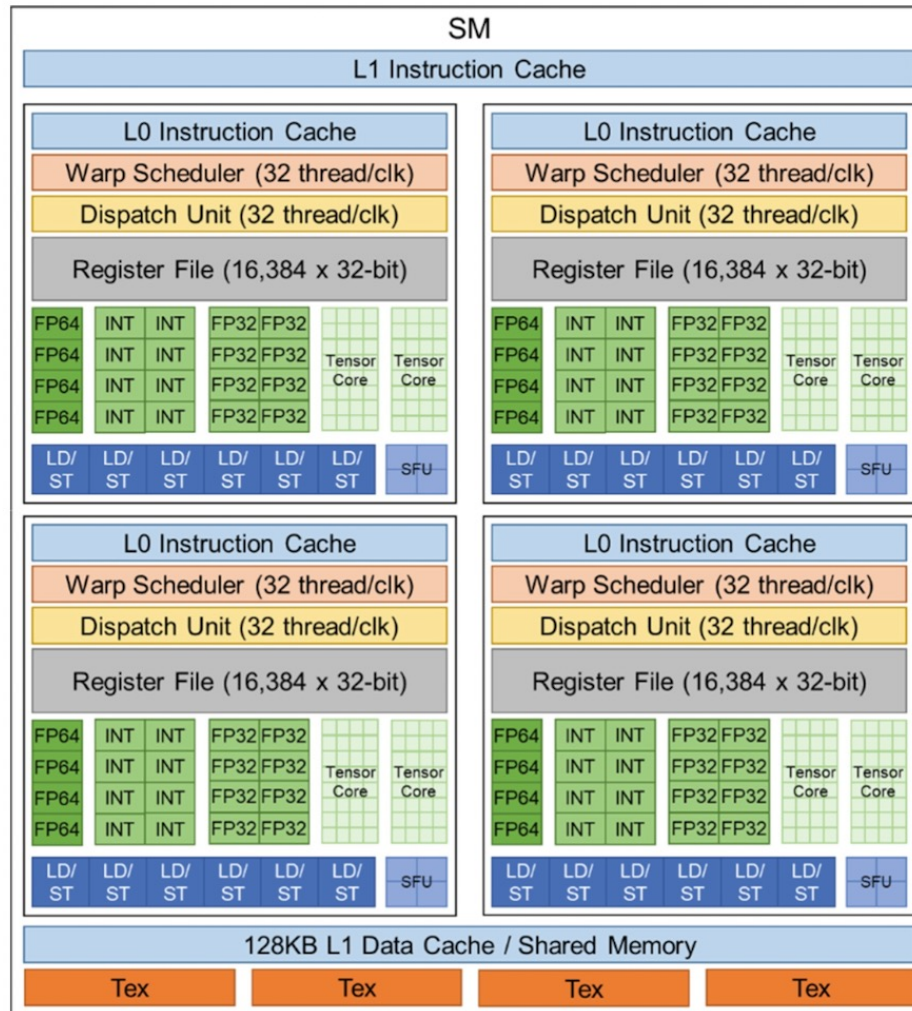
- These are the traditional GPU computing units
- Handle general-purpose parallel computing tasks
- Process standard graphics rendering workloads
- Execute regular FP32 (single precision) and INT32 operations
- Responsible for most traditional GPU workloads

Tensor Cores

- Specialized hardware accelerators designed specifically for matrix operations
- Optimized for deep learning and AI workloads
- Dramatically accelerate matrix multiply-accumulate operations (critical for neural networks)
- Can perform mixed-precision calculations (like FP16 and FP32)
- Provide order-of-magnitude speedups for deep learning training and inference
- Introduced with NVIDIA's Volta architecture and improved in subsequent generations

Spec	P100	V100	A100	H100	H200	B200
Architecture	Pascal	Volta	Ampere	Hopper	Hopper	Blackwell
SM count	56	80	108	132	132	148
CUDA cores/SM	64	64	128	128	128	128
Total CUDA cores	3,584	5,120	6,912	16,896	16,896	16,896
Tensor core gen	—	1st	3rd	4th	4th	5th
Tensor cores/SM	—	8	4	4	4	4
Total tensor cores	—	640	432	528	528	528
FLOPs/TC/cycle (FP16)	—	256	512	1,024	1,024	~2,048
Shared mem/SM	64 KB	96 KB	164 KB	228 KB	228 KB	228 KB
Register file/SM	256 KB	256 KB	256 KB	256 KB	256 KB	256 KB
Tensor mem (TMEM)/SM	—	—	—	—	—	256 KB
Warps/SM (max)	64	64	64	64	64	64
Tensor core precision	—	FP16, INT8	FP16, BF16, TF32, FP64, INT8	FP8, FP16, BF16, TF32, FP64, INT8	same as H100	FP4, FP6, FP8, FP16, BF16, TF32

V100



A100

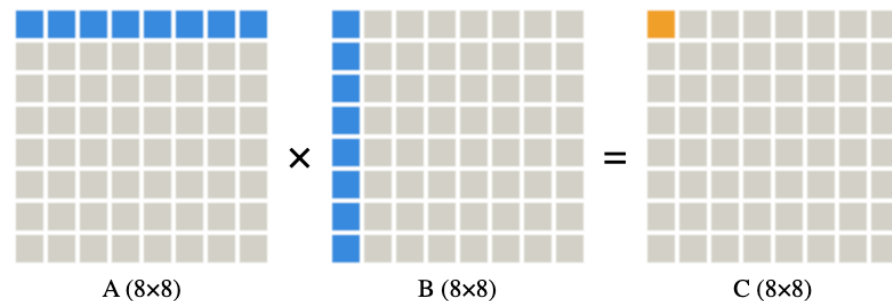


The memory wall

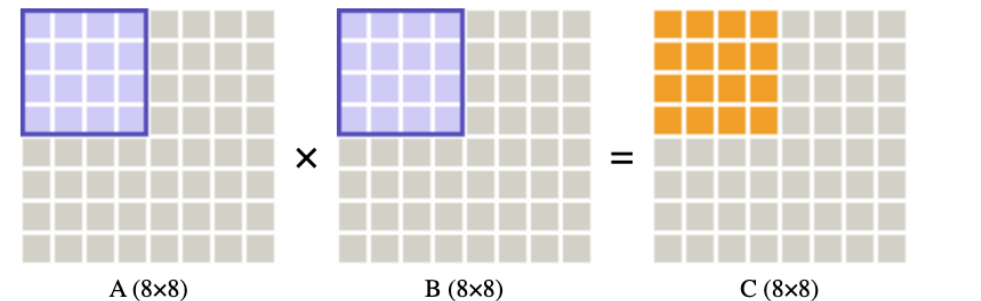
- **The memory wall: compute has outpaced bandwidth for 30 years**
 - H100: 67 TFLOP/s (FP32) but only 3.35 TB/s HBM3 bandwidth
 - Arithmetic intensity needed to stay compute-bound: $67 / 3.35 \approx 20$ FLOP/byte
 - Most real workloads fall below this, i.e., memory is the bottleneck
- **DRAM vs SRAM:**
 - SRAM (on-chip shared memory): ~1 TB/s, but only 228 KB per SM on H100.
 - DRAM (HBM3): 3.35 TB/s aggregate, but 80 GB, i.e., 3 orders of magnitude more.
 - Off-chip DRAM latency: ~100× slower than on-chip SRAM.
- **Energy cost of data movement (Horowitz, 2014):**
 - FP32 multiply: ~3.7 pJ
 - DRAM read (64-bit): ~640 pJ, i.e., roughly 170× more expensive.
 - Minimising DRAM traffic is therefore both a performance and energy objective.

Case study: Naïve vs tiled GEMM

Naïve GEMM



Tiled GEMM



- In **naïve GEMM**, computing each output element $C[i,j]$ requires loading the entire row i of A and column j of B from DRAM. Those same elements are loaded again for every other output element in the same row or column. No data is reused, i.e., total DRAM traffic scales as N^3 .
- In **tiled GEMM**, a $T \times T$ tile of A and a $T \times T$ tile of B are loaded once into shared memory (shown in purple). Every thread in the tile then computes against that shared data without going back to DRAM. **Each tile load is reused T times before the next tile is fetched.** Total DRAM traffic drops by a factor of T , which is exactly the arithmetic intensity improvement. I.e., you do the same number of MACs but move T times fewer bytes.
- **For your chiplet designs: this is the on-chip memory argument made concrete. Sizing the scratchpad or local SRAM to hold at least one full tile is what makes the difference between a memory-bound and a compute-bound design on the roofline.**
- Good explanation: <https://www.youtube.com/watch?v=Q3GgbfGTnVc>

Naive GEMM: DRAM traffic analysis

- **Naive matrix multiply ($M=N=K=64$, FP32):**
 - Each output element $C[i,j]$ requires: 1 row of A (64 floats) + 1 col of B (64 floats)
 - No reuse: every element loaded fresh from DRAM for each output
- **DRAM reads:**
 - A: $M \times K \times N$ reads = $64 \times 64 \times 64 = 262,144$ floats = 1,048,576 bytes (1 MB)
 - B: $N \times K \times M$ reads = $64 \times 64 \times 64 = 262,144$ floats = 1,048,576 bytes (1 MB)
 - Total DRAM traffic: 2 MB reads to produce 16 KB of output ($64 \times 64 \times 4$ bytes)
- **Arithmetic intensity:**
 - FLOPs: $2 \times 64 \times 64 \times 64 = 524,288$
 - Bytes: 2,097,152 (reads) + 16,384 (writes, negligible) ≈ 2.1 MB
 - AI = $524,288 / 2,097,152 \approx 0.25$ FLOP/byte, i.e., deep in memory-bound regime.

Tiled GEMM: DRAM traffic analysis

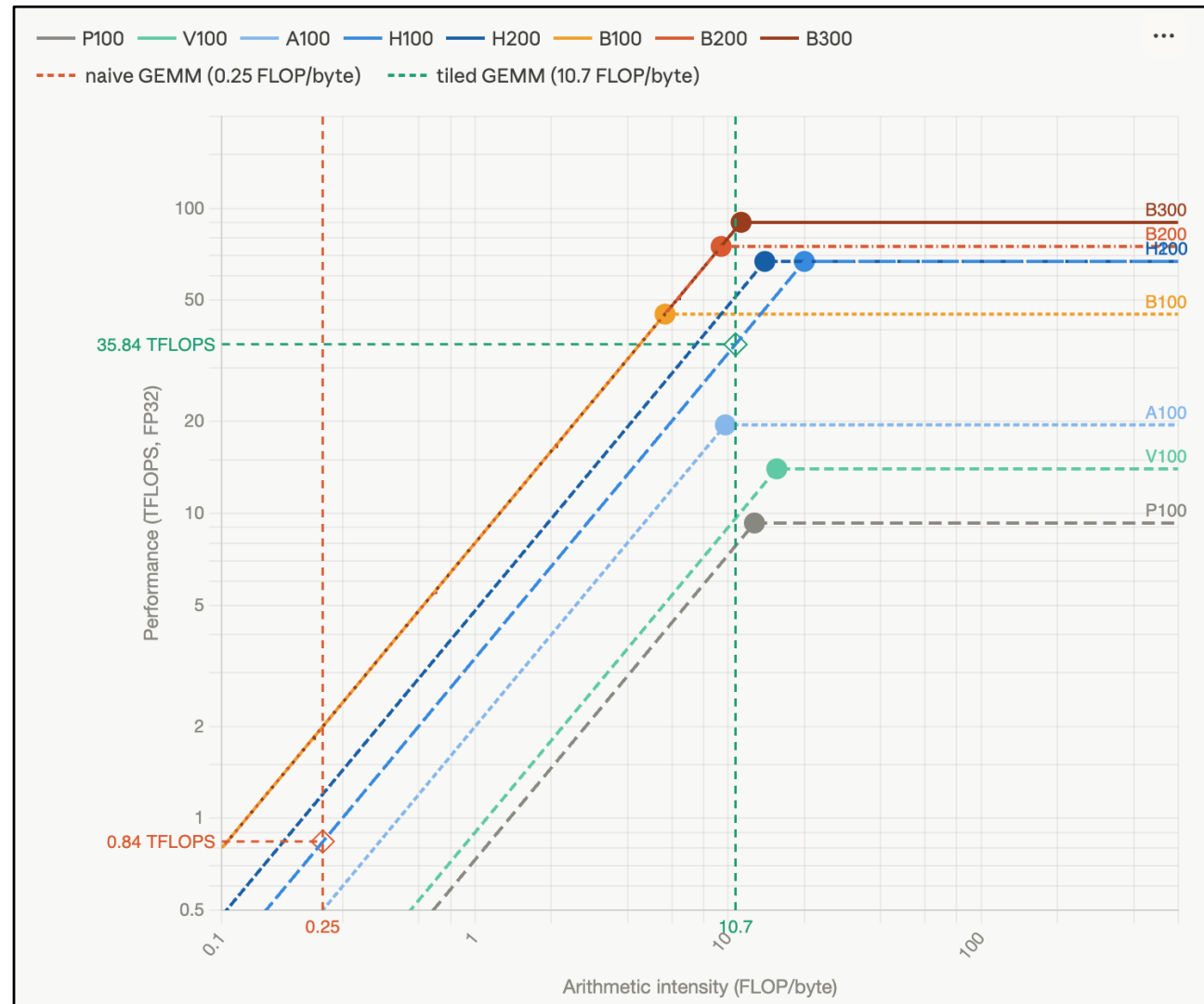
- **Tiled matrix multiply (tile size $T=16$, $M=N=K=64$, FP32):**
 - Load a $T \times T$ tile of A and $T \times T$ tile of B into shared memory
 - Each tile element is reused T times for dot products within the tile
- **DRAM reads:**
 - Every element of A and B is read from DRAM exactly once.
 - A: $M \times K$ floats loaded once = $64 \times 64 = 4,096$ floats = 16,384 bytes (16 KB)
 - B: $K \times N$ floats loaded once = $64 \times 64 = 4,096$ floats = 16,384 bytes (16 KB)
 - Total DRAM traffic: 32 KB, i.e., **a 64× reduction over naive** (= tile size $T=16^2/4$)
- **Arithmetic intensity:**
 - FLOPs unchanged: 524,288
 - Bytes: 32,768 (reads) + 16,384 (writes) $\approx$ 49 KB
 - $AI = 524,288 / 49,152 \approx 10.7$ FLOP/byte, i.e., approaching ridge point on H100.
- **Larger tiles $\rightarrow$ higher reuse $\rightarrow$ closer to compute-bound. Limit: shared memory size.**

Roofline placement: naive vs. tiled GEMM

- **H100 roofline reference points:**
 - Peak FP32 compute: 67 TFLOP/s
 - Peak High Bandwidth Memory 3 (HBM3) : 3.35 TB/s
 - Ridge point: $67\text{e}12 / 3.35\text{e}12 \approx 20$ FLOP/byte
- **Naive 64×64 GEMM: AI ≈ 0.25 FLOP/byte**
 - 80× below the ridge point, i.e., entirely memory-bound
 - Attainable performance: $0.25 \times 3.35\text{e}12 \approx 0.84$ TFLOP/s (1.25% of peak)
- **Tiled 64×64 GEMM (T=16): AI ≈ 10.7 FLOP/byte**
 - Still below ridge point but 43× better than naive
 - Attainable performance: $10.7 \times 3.35\text{e}12 \approx 35.8$ TFLOP/s (53% of peak)
- **Lesson: tiling moves the operating point rightward on the roofline.**
 - This is what you will measure in Nsight on Wednesday.

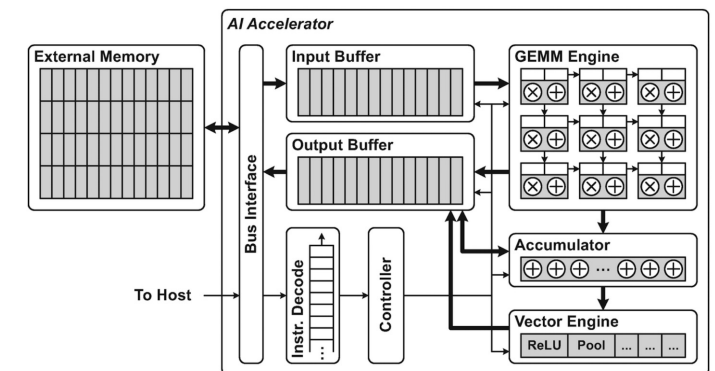
Roofline placement: naive vs. tiled GEMM

Lesson: tiling
moves the
operating point
rightward on
the roofline.



Project: Accelerator datapath template

- **Use the general AI accelerator block diagram as a project template:**
 - Host interface: AXI4-Lite (control) + AXI4-Stream (data) — your M1 choice.
 - Input buffer: on-chip SRAM sized to one tile of your dominant kernel
 - Compute engine: the kernel-specific datapath (MAC array, dot-product unit, etc.)
 - Accumulator: partial sums before writing back to output buffer
 - Output buffer: holds results before DMA to host
 - Controller: FSM driving tile iteration and interface transactions
- **Design questions to answer for your block diagram:**
 - What is your dominant kernel? (from profiling)
 - What tile size fits in your on-chip SRAM budget?
 - What precision does your compute engine use? (FP32 / INT8 / BF16)
 - Is your kernel compute-bound or memory-bound at that tile size?



Project: HW/SW partitioning decision criteria

- **Three criteria for deciding what to accelerate in hardware:**
 - 1. Does the kernel dominate total runtime? (From profiling — Amdahl's law applies).
 - 2. Is the kernel compute-bound or memory-bound at the target tile size?
 - 3. Does the kernel have regular enough data access patterns to map to a fixed datapath?
- **Compute-bound → good candidate: parallelize the arithmetic**
 - Dense GEMM, convolution, dot products over large vectors
- **Memory-bound → harder: need to restructure data access first**
 - Sparse ops, embedding lookups, graph message passing
 - Must show tiling or fusion closes the gap before committing to HW
- **Irregular access patterns → reconsider scope**
 - If data-dependent branching or pointer chasing, a fixed datapath will stall
 - This is a common over-scoping failure mode in the project

CUDA

- **CUDA** (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA.
- It enables developers to use NVIDIA graphics processing units (GPUs) for general-purpose computing, a practice known as **GPGPU** (General-Purpose computing on Graphics Processing Units).



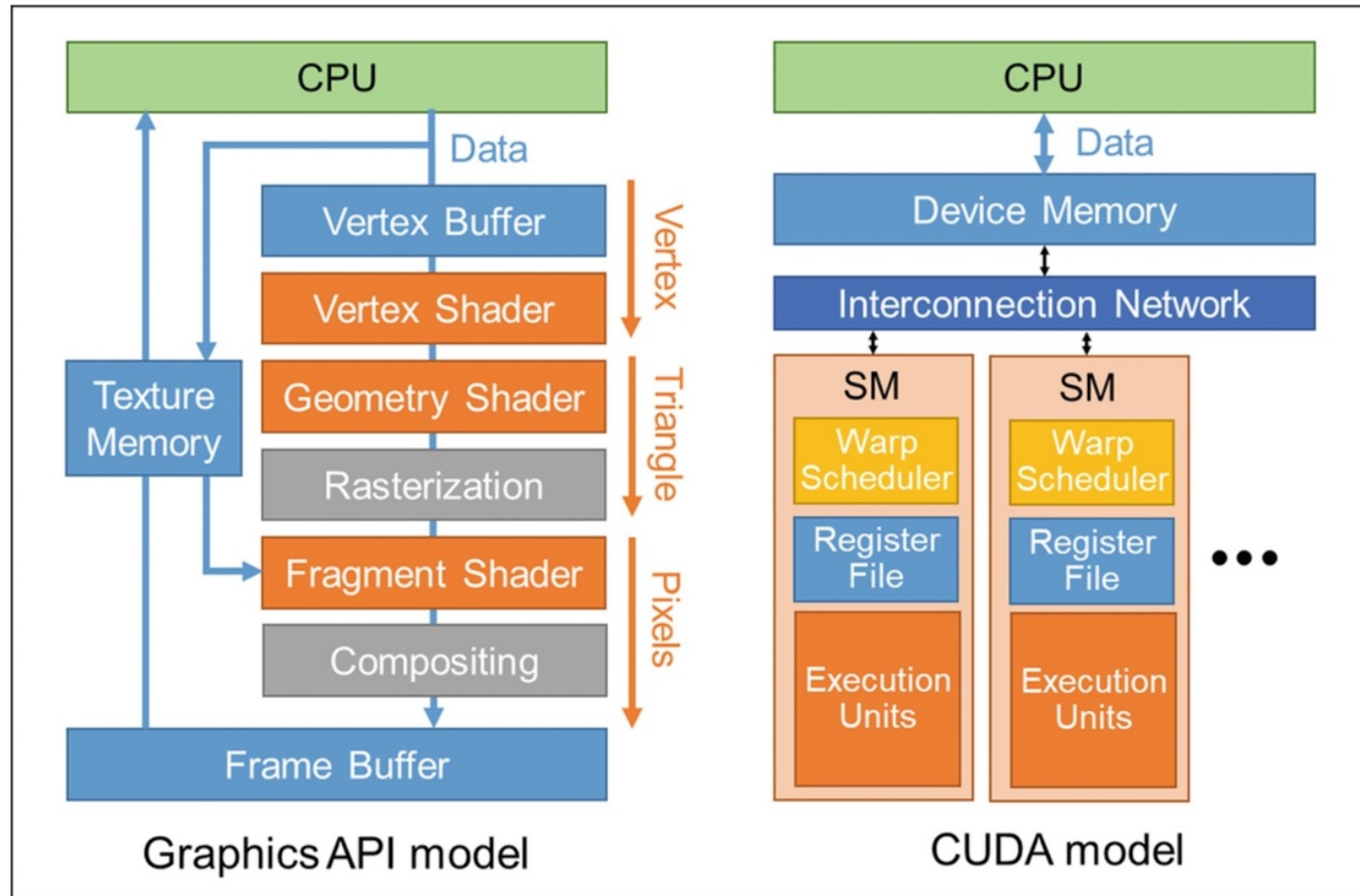
- Good resource: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>

CUDA model

Good resource: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>



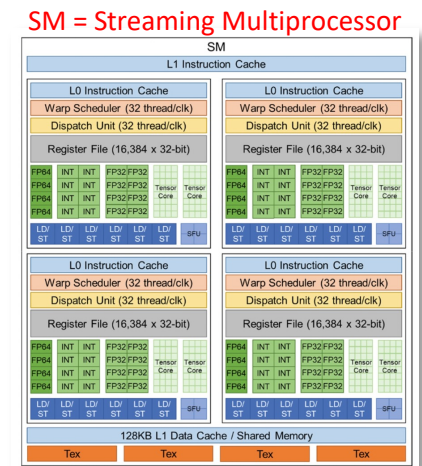
CUDA (**Compute Unified Device Architecture**) is a parallel computing platform and programming model created by NVIDIA that allows software developers to use NVIDIA GPUs for **general-purpose processing**, known as **GPGPU**



Early GPGPU was implemented by formulating data and operations in graphics primitives and translated to pass through a fixed graphics pipeline.

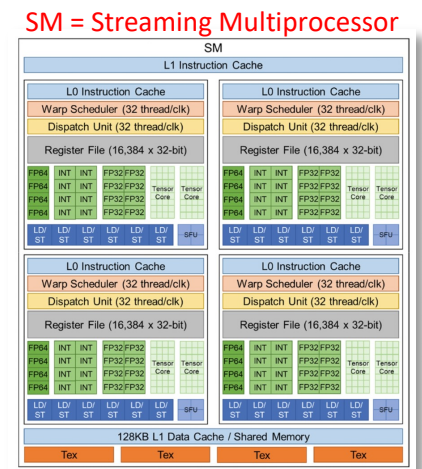
Warp

- A "warp" is a group of threads that execute instructions simultaneously on a GPU (Graphics Processing Unit).
- The warp scheduler is a component of the GPU architecture that manages the execution of these thread groups.
- **A warp typically consists of 32 threads in NVIDIA GPUs** (AMD calls similar groupings "wavefronts"). These threads execute the same instruction at the same time in a **SIMT** (Single Instruction, Multiple Thread) architecture.
- The warp scheduler is responsible for:
 - Deciding which warps are ready to execute.
 - Allocating compute resources to warps.
 - Managing warp context switching when threads need to wait for memory operations.
 - Hiding memory latency by switching between warps.
- **Warp divergence (key failure mode)**
 - ☐ If threads in a warp take different branches (if/else), the GPU serializes both paths.
 - ☐ Threads on the idle path are masked off. Throughput drops by up to 2x per branch level.
 - ☐ Avoid control flow that depends on thread ID. Keep all 32 threads on the same path.



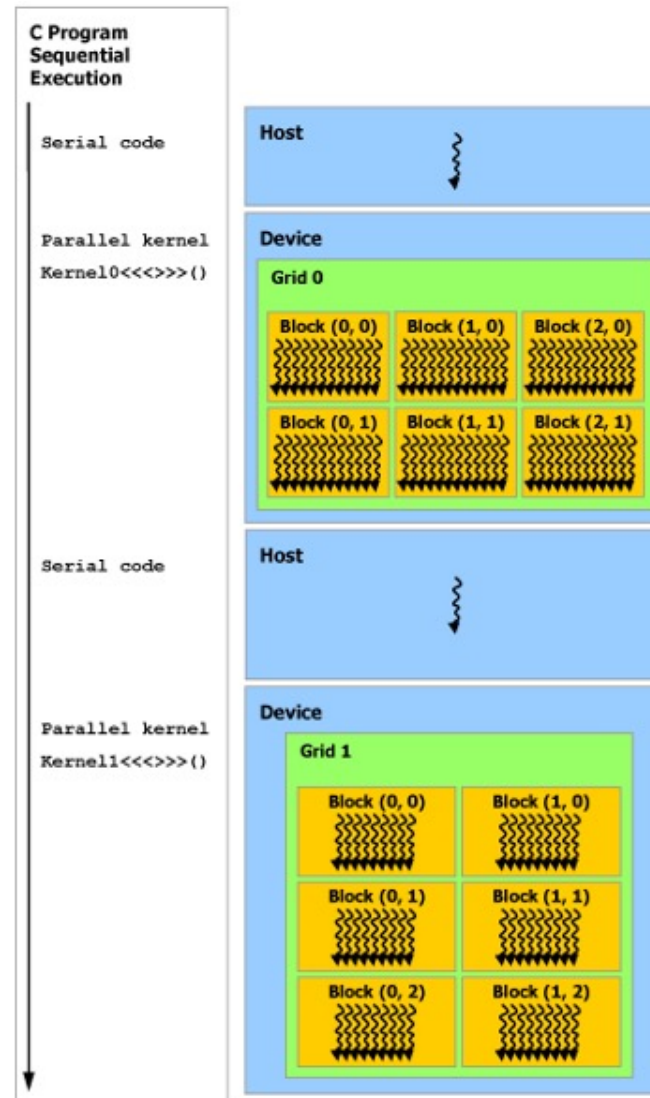
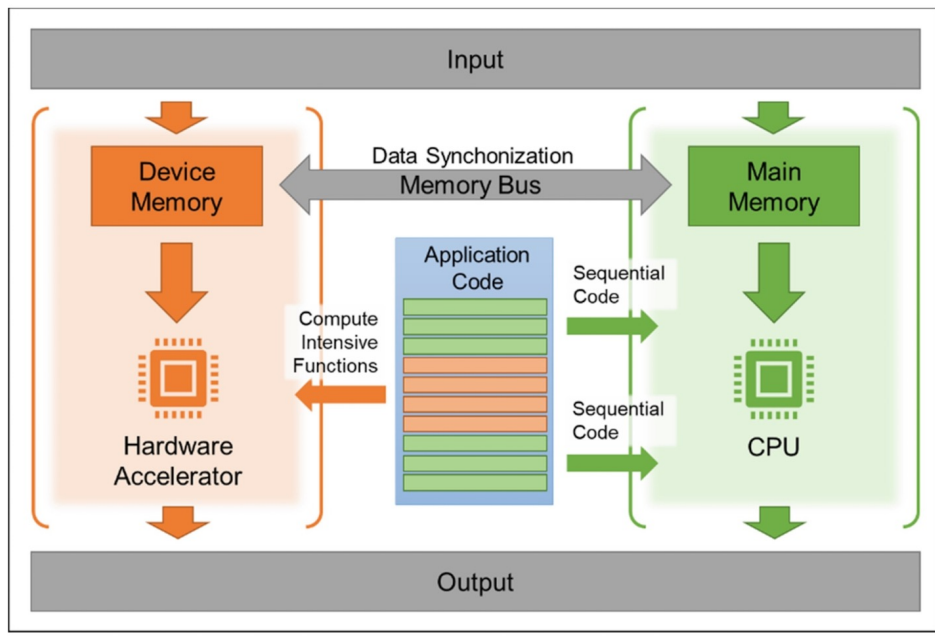
CUDA software → GPU hardware mapping

- **CUDA abstraction maps directly onto GPU hardware:**
 - CUDA thread → executes on a single CUDA core (FP32 unit)
 - Warp (32 threads) → scheduled as one unit by the warp scheduler; executes one instruction in SIMT lockstep
 - Thread block → assigned to one SM; shares that SM's shared memory
 - Grid of blocks → distributed across all SMs on the GPU
- **Implication for performance:**
 - Shared memory is per-SM: blocks cannot share data across SMs
 - Global memory (DRAM) is accessible by all threads but is slow
 - Maximising SM occupancy = keeping the warp scheduler busy
 - Warp divergence (branching within a warp) serializes lanes → avoid if/else on thread ID within a warp
- **Occupancy = active warps / max warps per SM**
 - Limited by register count, shared memory usage, and block size
 - Low occupancy → memory latency not hidden → idle compute units



Programming and execution model

A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A **multithreaded program partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

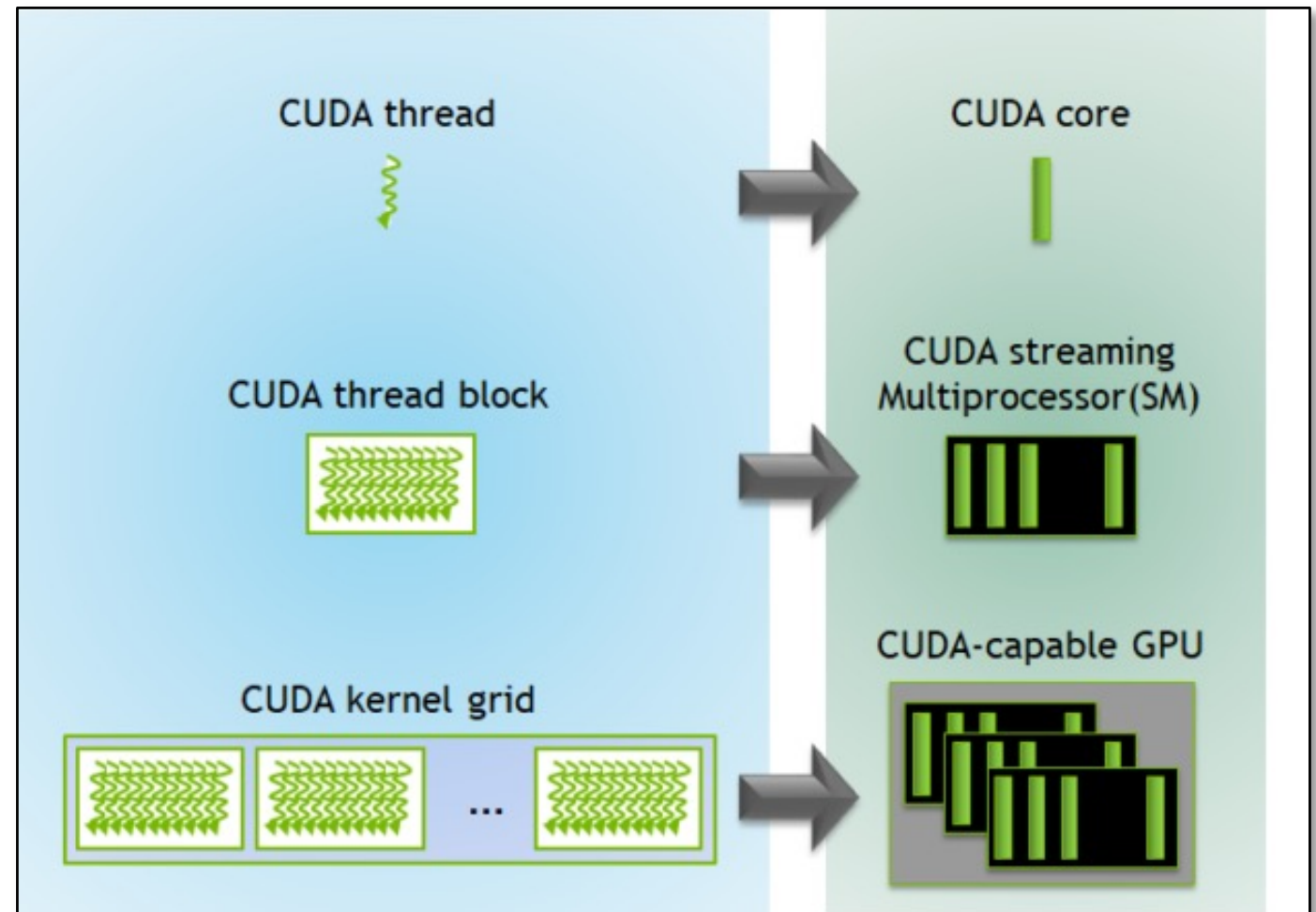
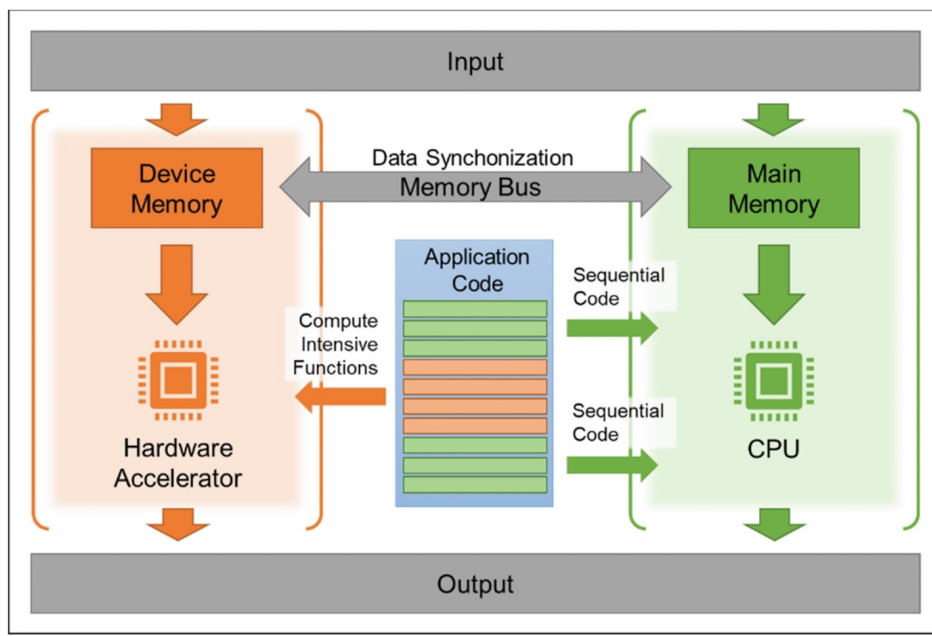


CUDA programming model assumes that the **CUDA threads execute on a physically separate device** that operates as a coprocessor to the host running the C++ program.

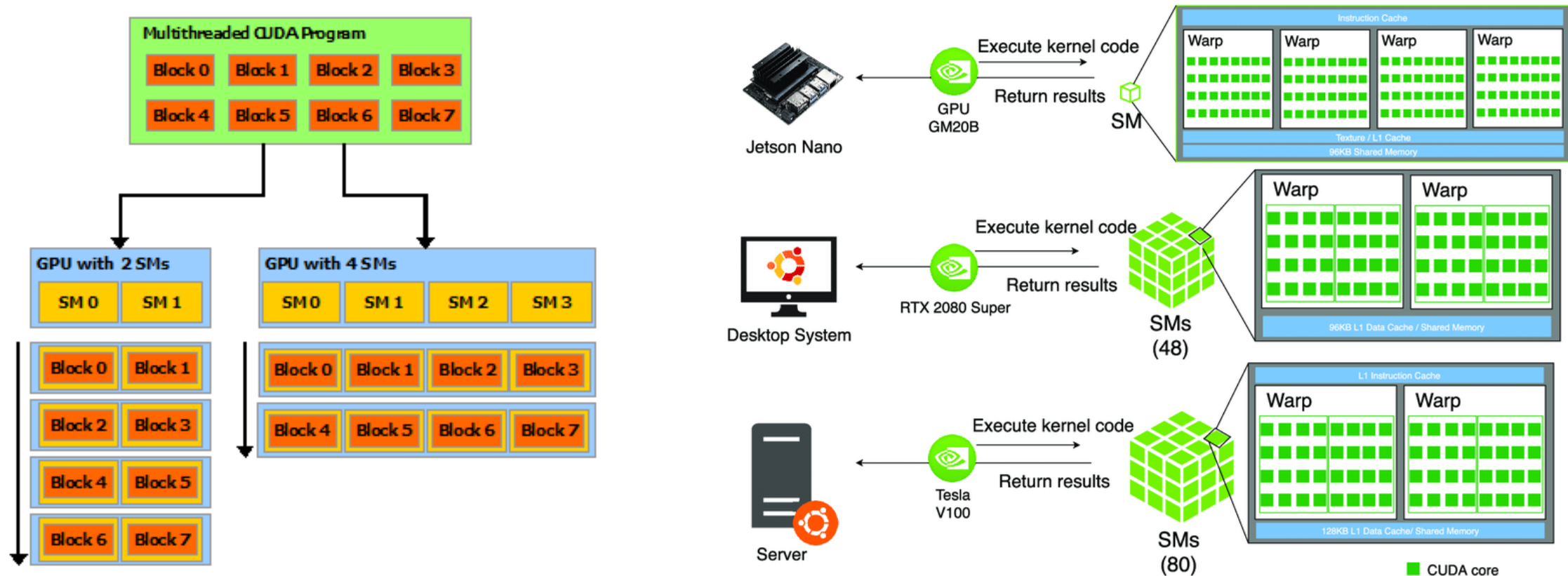
Serial code executes on the host while parallel code executes on the device.

Programming and execution model

A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A multithreaded program is **partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.



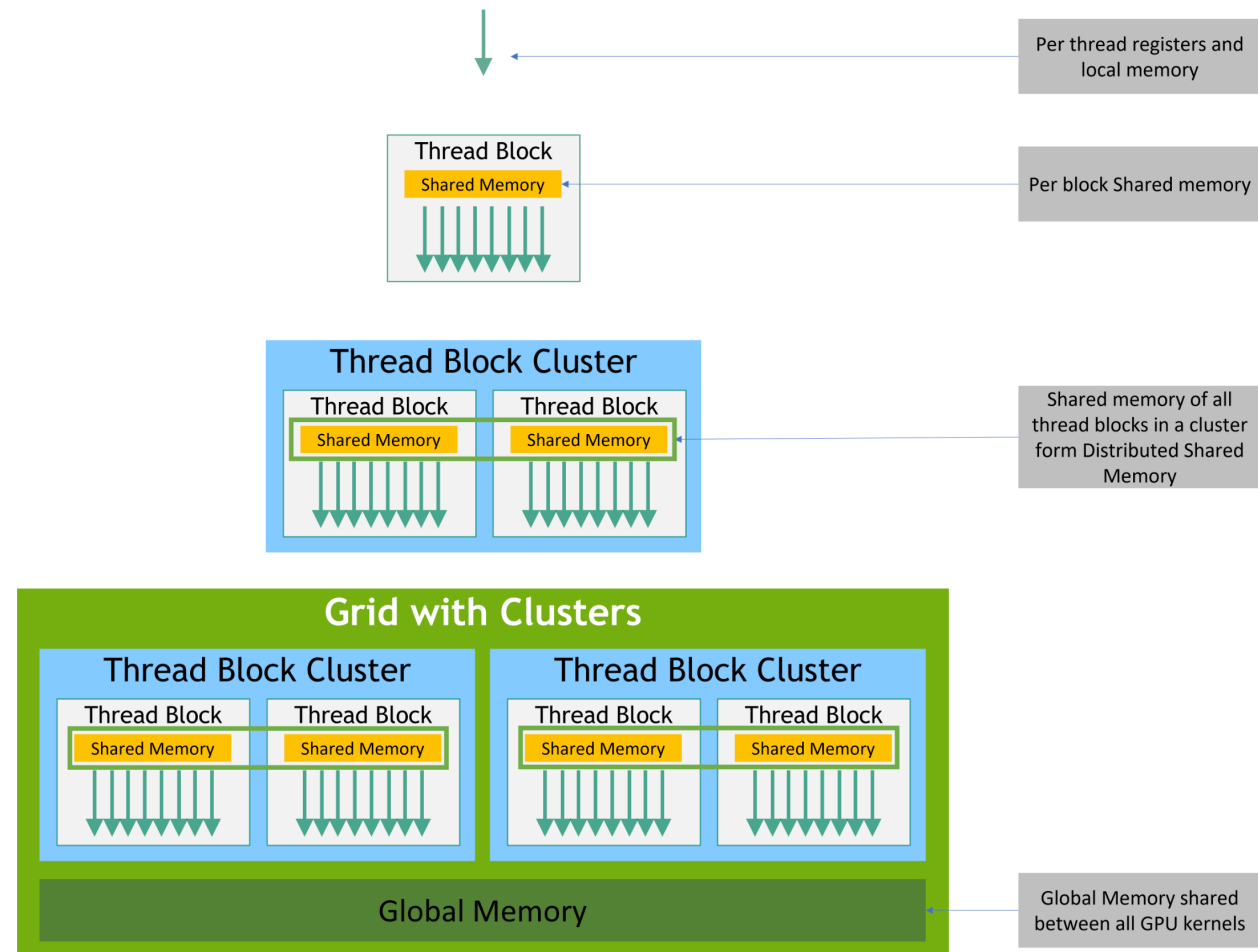
Automatic scalability



A GPU is built around an **array of Streaming Multiprocessors (SMs)**. A **multithreaded program is partitioned into blocks of threads that execute independently from each other**, so that a GPU with more multiprocessors will automatically execute the program in less time than a GPU with fewer multiprocessors.

Memory hierarchy

- CUDA threads may access data from multiple memory spaces during their execution.
- **Each thread has private local memory.**
- **Each thread block has shared memory visible to all threads of the block and with the same lifetime as the block.**
- Thread blocks in a thread block cluster can perform read, write, and atomics operations on each other's shared memory.
- **All threads have access to the same global memory.**



Kernels (1)

- CUDA C++ extends C++ by allowing the programmer to define C++ functions, called **kernels**, that, when called, **are executed N times in parallel by N different CUDA threads, as opposed to only once like regular C++ functions.**
- A kernel is defined using the **__global__ declaration specifier** and the number of CUDA threads that execute that kernel for a given kernel call is specified using a new **<<<...>>> execution configuration** syntax.
- Each thread that executes the kernel is given a unique thread ID that is accessible within the kernel through built-in variables.

```
#include <iostream>

__global__ void kernel( void ) {

}

int main( void ) {
    kernel<<<1,1>>>();
    printf( "Hello, World!\n" );
    return 0;
}
```

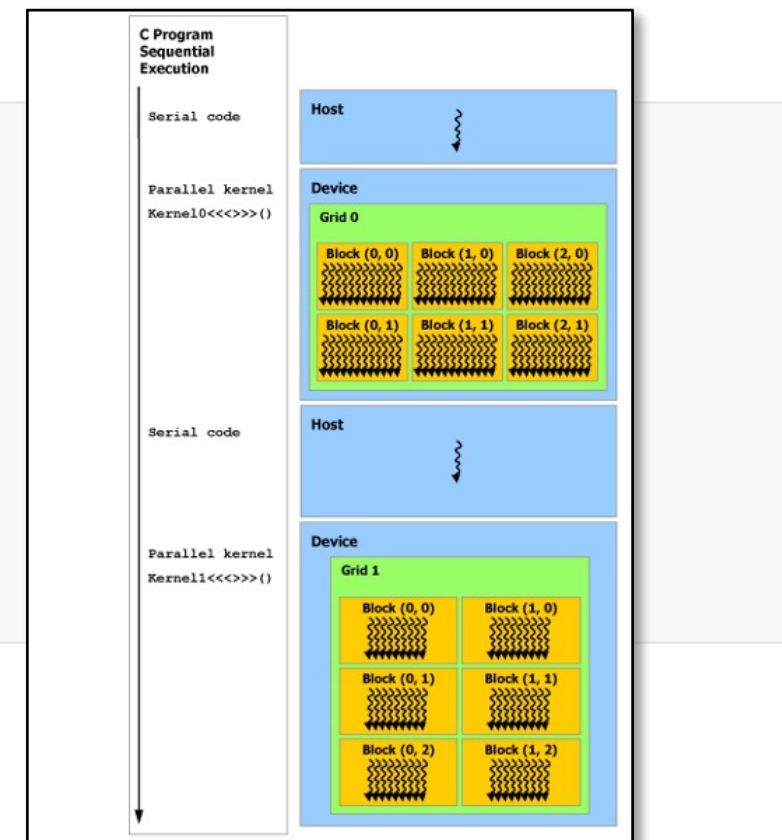
Kernels (2)

- As an illustration, the following sample code, using the built-in variable threadIdx, adds two vectors A and B of size N and stores the result into vector C.
- Single Instruction, Multiple Threads (SIMT)**

```
// Kernel definition
__global__ void VecAdd(float* A, float* B, float* C)
{
    int i = threadIdx.x;
    C[i] = A[i] + B[i];
}

int main()
{
    ...
    // Kernel invocation with N threads
    VecAdd<<<1, N>>>(A, B, C);
    ...
}
```

Here, each of the N threads that execute `VecAdd()` performs one pair-wise addition.





Christof Teuscher



teuscher@pdx.edu

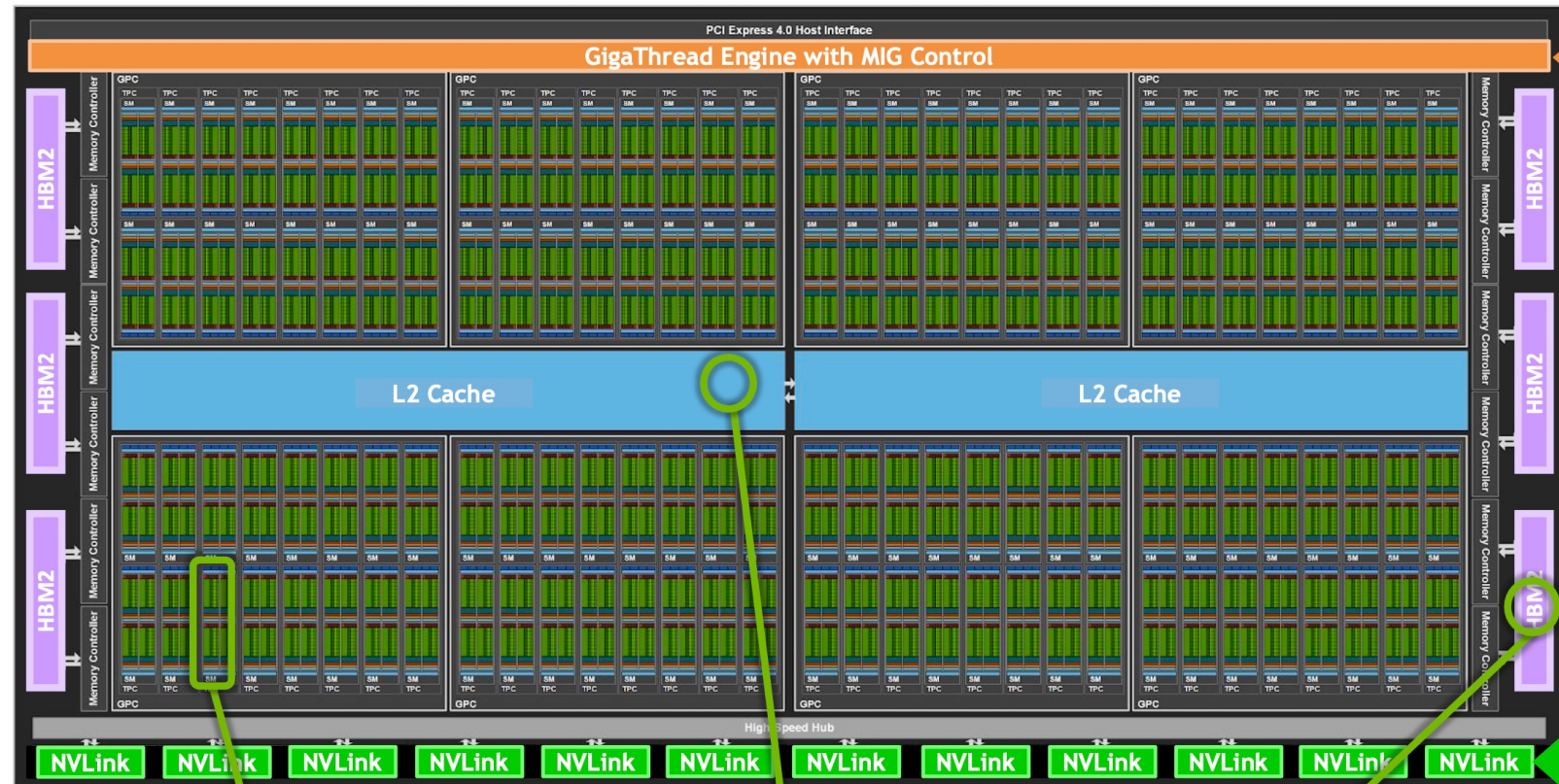
www.teuscher-lab.com/teaching



GPU Memory Hierarchy

A100 TENSOR-CORE GPU

54 billion transistors in 7nm



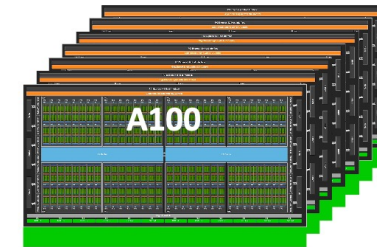
108 SMs
6912 CUDA Cores

40MB L2
6.7x capacity

1.56 TB/s HBM2
1.7x bandwidth

Scale OUT

Multi-Instance GPU

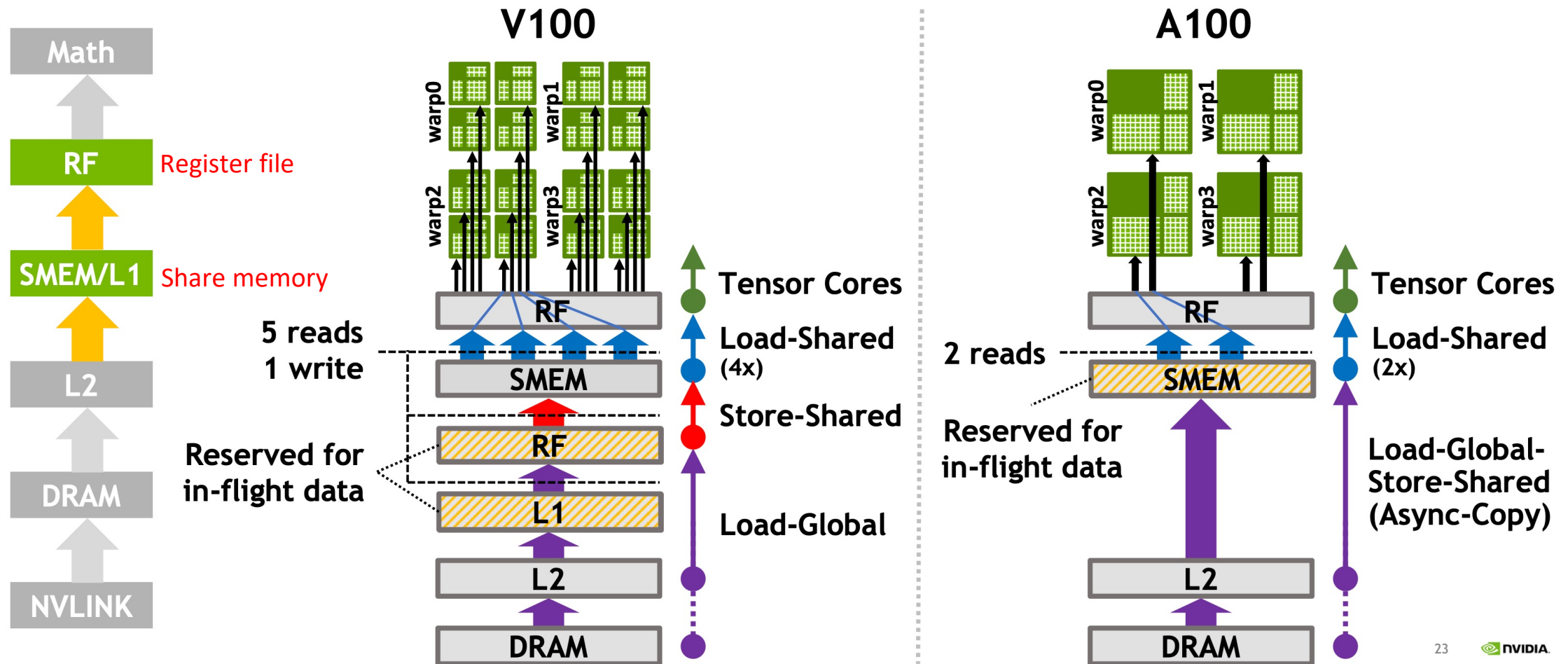


2x BW
Scale UP

3rd gen.
NVLINK

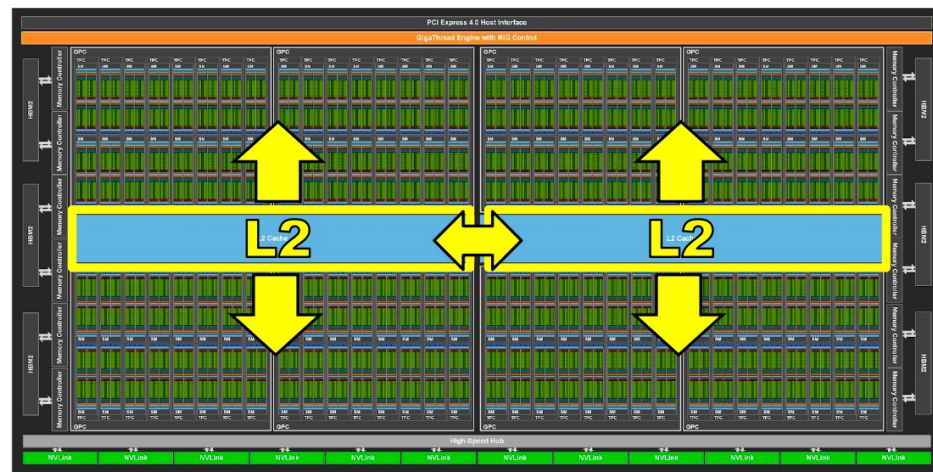
A100 data movement efficiency

3x SMEM/L1 bandwidth, 2x in-flight capacity



A100 data movement efficiency

Split L2 with
hierarchical crossbar -
2.3x increase in
bandwidth over V100,
lower latency

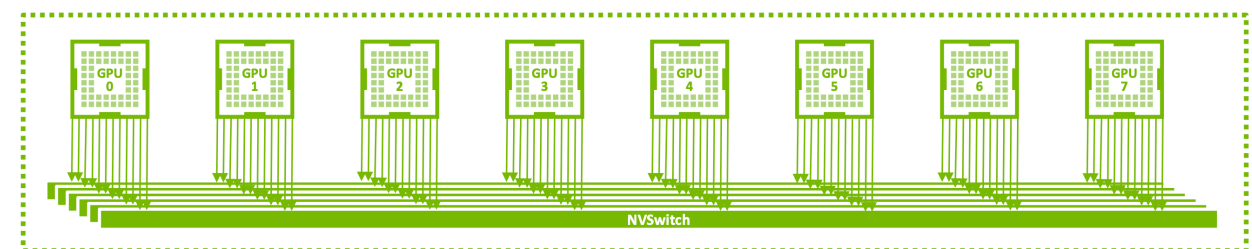
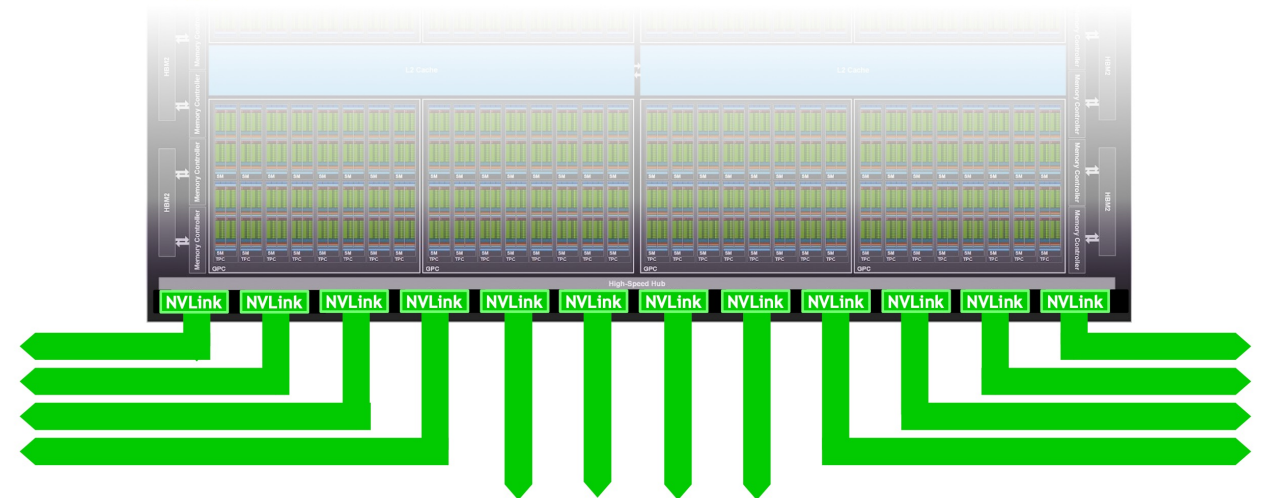


Third Generation NVLink

50 Gbit/sec per signal pair

12 links, 25 GB/s in/out, 600 GB/s total

2x vs. V100



Hardware consistency